TECHNICAL DOCUMENTATION

AppPanrom

A Tanglish Programming Language and Android App Development Studio

How it works, what happens behind the scenes, and why we built it.

TEAM

Kokkila (Team Leader)
Jothika
G. Naveen Shankar
P. Santhosh
G. Giri
M. Shalom Arputha Singh

Anand Institute of Higher Technology
Version 1.0 | November 2026

Contents

1 Introduction 4

1.1 What AppPanrom is 4

1.2 Why we created it 4

1.3 Goals and non-goals 5

1.4 The project at a glance 5

2 Product Overview 6

2.1 The user journey 6

2.2 Feature summary 7

2.3 A first program 7

3 System Architecture 8

3.1 The big picture 8

3.2 Components and responsibilities 8

3.3 Data flow of a single keystroke 9

3.4 Repository layout 9

3.5 Technology choices 9

4 The AppPanrom Language 10

4.1 Design philosophy 10

4.2 Lexical structure 10

4.3 Values and variables 11

4.4 Control flow 11

4.5 Functions and handlers 11

4.6 Screens and components 11

4.7 Styling 12

4.8 The standard library 13

4.9 What is deliberately missing 13

5 The Compiler 13

5.1 Stage 1: the lexer 13

5.2 Stage 2: the parser 15

5.3 Stage 3: semantic validation and lowering 17

5.4 The shared specification 20

5.5 Stage 4: the intermediate representation 20

5.6 Diagnostics 20

6 The Runtime 21

6.1 The execution model 21

6.2 Values and safety helpers 21

6.3 Limits and errors 22
6.4 Evaluating the IR 22
6.5 Pure rendering 23
6.6 Building components 24
6.7 Local storage 24
6.8 The DOM renderer 25

7 The Visual Designer 25
7.1 The two-way loop 25
7.2 Addressing components by source line 25
7.3 Operations 26
7.4 Never write invalid code 28
7.5 A real edit 28
7.6 The inspector and the canvas 28

8 The Backend 29
8.1 The HTTP API 29
8.2 Uniform errors 30
8.3 Projects and files 30
8.4 The database 31
8.5 Configuration 32

9 The Studio (Frontend) 32
9.1 Layout and panels 32
9.2 The editor 32
9.3 Save, validate and the sync state 33
9.4 The sandboxed preview 33
9.5 The design canvas 34
9.6 The build panel 35

10 The Android Pipeline 35
10.1 The template 35
10.2 The generator 37
10.3 Prerequisite detection 38
10.4 The build manager 38
10.5 Verifying the result 40
10.6 Release signing 40
10.7 App icon 41
10.8 Continuous integration: real builds 41

11 Security Model 42
11.1 What is protected, and how 42
11.2 Known limits of the model 42

12 Testing and Quality 43
12.1 The Gradle test double 43
12.2 Testing real apps, not just features 44
12.3 What the tests caught 44
13 Example Apps 44
13.1 Patterns the examples use 45
13.2 Design notes for the example apps 46
14 Limitations and Future Work 46
14.1 Honest status 46
14.2 Known limitations 46
14.3 Future work 47
A Appendix: Reference Tables 47
A.1 Built-in functions 47
A.2 Configuration functions (top level only) 48
A.3 Common error messages 48
A.4 Running the project 49
B Appendix: Glossary 49

1 Introduction

1.1 What AppPanrom is

AppPanrom is a complete, small-scale toolchain for building Android apps in a programming language whose keywords are written in Tanglish (Tamil written in English letters). It has three parts that work as one product:

- **The AppPanrom language.** A deliberately small language with Tanglish keywords such as `mari` (declare a variable), `enna_na` (if), `illana` (else), `ovvoru` (for each) and `screen_uruvakku` (create a screen). Programs describe state, behaviour and screens.
- **AppPanrom Studio.** A local, web-based IDE with a code editor, live diagnostics, a phone-shaped live preview, a drag-and-drop visual designer, a project dashboard and a build panel.
- **The Android pipeline.** A generator and build manager that turn the same program into a real Gradle-based Android project, build an installable APK (or a signed release APK / AAB), verify the result and report the outcome honestly.

A learner writes a few lines, presses Run and immediately sees the app in a phone frame. When the app works, one click produces an APK that can be installed on a real phone. Nothing in this path requires the learner to know Kotlin, Java, Gradle or Android Studio.

1.2 Why we created it

Learning to build a first app is harder than it should be. A typical beginner faces several barriers at once, and each one is a reason to give up before the first button ever appears on screen:

Barrier	What the beginner meets	How AppPanrom removes it

Heavy tooling	Android Studio, the SDK, Gradle, Kotlin or Java, emulators; hours of installation and configuration.	The Studio runs in a browser. The preview needs no SDK. The SDK is only needed for the final APK step.
Language barrier	All mainstream languages use English keywords and English error messages, a hurdle for many first-time learners.	Tanglish keywords and plain-language error messages with suggestions.
Disconnected steps	Layout in one tool, logic in another, build in a third. A change in one place breaks another.	One source file is the single truth. The designer and the editor edit the same text.
Unclear failures	Cryptic compiler output, long Gradle stack traces.	Diagnostics with line, column and a suggested fix. Build failures say which prerequisite is missing.

WHY TANGLISH

Tanglish is how many Tamil speakers already type and talk about technology. Writing `enna_na count > 0:` reads naturally to them: *enna na* means "what if". Lowering the language barrier lets a learner concentrate on the actual ideas of programming: state, conditions, loops, functions and user interfaces. The language is intentionally small so that the whole of it can be learned in an afternoon.

1.3 Goals and non-goals

Goals

- A learner can go from an empty project to a running app in minutes, with immediate visual feedback.
- The preview and the final APK behave the same, by construction (one shared runtime, Chapter 6).
- Design and code never disagree: the designer edits the source text, and the code redraws the designer (Chapter 7).
- User programs are untrusted. They are interpreted with limits; they are never turned into Python or JavaScript and never executed with `eval` or `exec` (Chapter 11).
- Honest reporting. A build is reported as successful only when a real, verified artifact exists (Chapter 10).
- Local-first. Projects and builds live on the developer's machine; no account or cloud service is needed.

Non-goals (in this version)

- It is not a general-purpose language and does not try to replace Kotlin or Flutter.
- Network APIs, camera, location and other runtime-permission device features are not implemented. The compiler tells the user so when they are used (Section 4.9).
- It is a single-user local tool with no authentication. It must not be exposed to the internet as is (Chapter 11).

1.4 The project at a glance

Item	Value
Backend	Python 3.11+ (developed on 3.13), FastAPI, SQLite. About 3,100 lines across the language engine, API, build manager and Android generator.
Runtime	One dependency-free JavaScript file of 1,205 lines: interpreter, view-tree builder, DOM renderer and design mode.
Studio	React 18, TypeScript, Vite, CodeMirror 6. Fourteen React components plus a typed API client.

Android template	Android Gradle Plugin 8.5.2, compileSdk and targetSdk 34, minSdk 24, a single Activity hosting a WebView, one dependency (`androidx.webkit`).
Shared specification	`spec/schema.json` (889 lines): components, style keys, standard library, permissions, not-yet-implemented names.
Language surface	18 UI components, 47 built-in functions, 5 app-configuration functions, 19 style keys, 18 reserved words.
Automated tests	156 unit and integration tests (120 backend, 36 runtime) plus a browser end-to-end suite.
Example apps	9 programs, 7 of them built as real APKs by continuous integration.

2 Product Overview

2.1 The user journey

1. **Create a project.** The dashboard creates a folder with a starter `main.apn` and a database row holding the name, application id, version and theme.
2. **Write code.** The editor highlights Tanglish keywords, built-in functions, strings and comments. Every 450 ms after typing stops, the source is compiled on the server and the problems appear as underlines and in the Diagnostics tab.
3. **Run.** Compilation produces an intermediate representation (IR). The Studio renders it in a phone frame with the same runtime that ships inside the APK.
4. **Design.** In design mode the user drags components from a palette onto the phone, selects and edits them in an inspector, and the code in the editor changes at the same moment.
5. **Build.** The Build tab checks prerequisites, generates an Android project from the template, runs Gradle in the background, streams the log, verifies the produced APK and offers it for download.

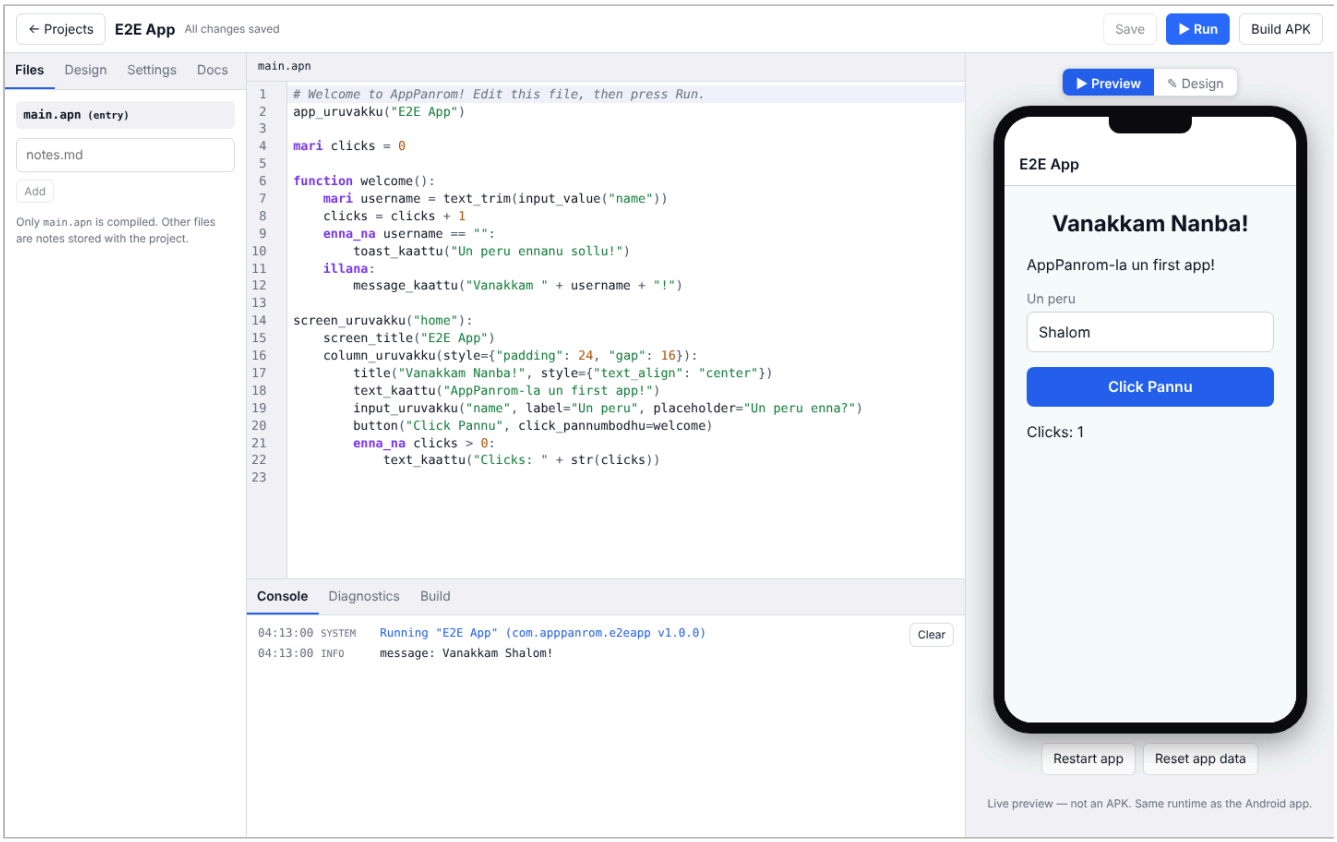


Figure 2.1: The Studio. File list on the left, editor in the centre, live phone preview on the right, console below.

2.2 Feature summary

Area	Features
Language	Variables, numbers, text, booleans, lists, dictionaries; arithmetic, comparison and logic operators; `enna_na / illana_enna / illana`; `ovvoru` and `irukkura_varai` loops; functions with return values; `try / catch`; multi-screen navigation; styling; local data storage.
Diagnostics	Errors and warnings with file, line, column, an error code and a suggestion ("Did you mean 'text_kaattu'?").
Studio	Project dashboard, multi-file projects, autosave, syntax highlighting, error underlines, console, settings, built-in language reference with examples.
Visual designer	Drag and drop from a palette, move, duplicate, delete, outline tree, inspector for properties, events and styles, add screens and handlers, undo and redo.
Android	Debug APK, release APK and release AAB; custom app icon; automatic permission declaration; signing from environment variables; independent signature verification.
Quality	Automated tests, continuous integration that builds example apps with the real Android SDK and Gradle.

2.3 A first program

The smallest useful app declares a variable, a handler function and one screen. This is the shipped example `examples/counter.apn` (abridged):

examples/counter.apn (lines 1-27) - Counter example

```
# Counter - global variables are the app state; the screen redraws after every event.
app_uruvakku("Counter")
app_id_set("com.apppanrom.counter")

mari count = 0

function plus():
    count = count + 1

function minus():
    enna_na count > 0:
        count = count - 1
    illana:
        toast_kaattu("Count 0-ku keela poga mudiyaadhu")

function reset():
    count = 0

screen_uruvakku("home", style={"background": "#EEF2FF"}):
    screen_title("Counter")
    center_ama(style={"padding": 24, "gap": 20}):
        text_kaattu("Count: " + str(count), style={"font_size": 36, "font_weight": "bold", "text_color": "#312E81"})
        progress_kaattu(min(count * 10, 100), label="Progress to 10")
        row_uruvakku(style={"gap": 12}):
            button("-", click_pannumbodhu=minus, style={"background": "#EF4444", "corner_radius": 24, "width": 64})
            button("+", click_pannumbodhu=plus, style={"background": "#10B981", "corner_radius": 24, "width": 64})
        button("Reset", click_pannumbodhu=reset, style={"background": "#6B7280"})
```

The top-level statements `app_uruvakku` and `app_id_set` configure the app. `mari count = 0` declares a global variable, which is the app's state. `plus`, `minus` and `reset` are handler functions. `screen_uruvakku("home")` declares a screen whose body is a tree of components. Every time an event handler finishes, the screen body is evaluated again with the new variable values and the display is rebuilt. This model (state in variables, screen as a pure function of state) is described in detail in Section 6.5.

3 System Architecture

3.1 The big picture

AppPanrom is built around one idea: **compile once into a data structure, render that data structure with one shared runtime in two places**. The data structure is the intermediate representation (IR), a JSON document that describes the app completely. The runtime is a single JavaScript file that interprets the IR and draws the screens.

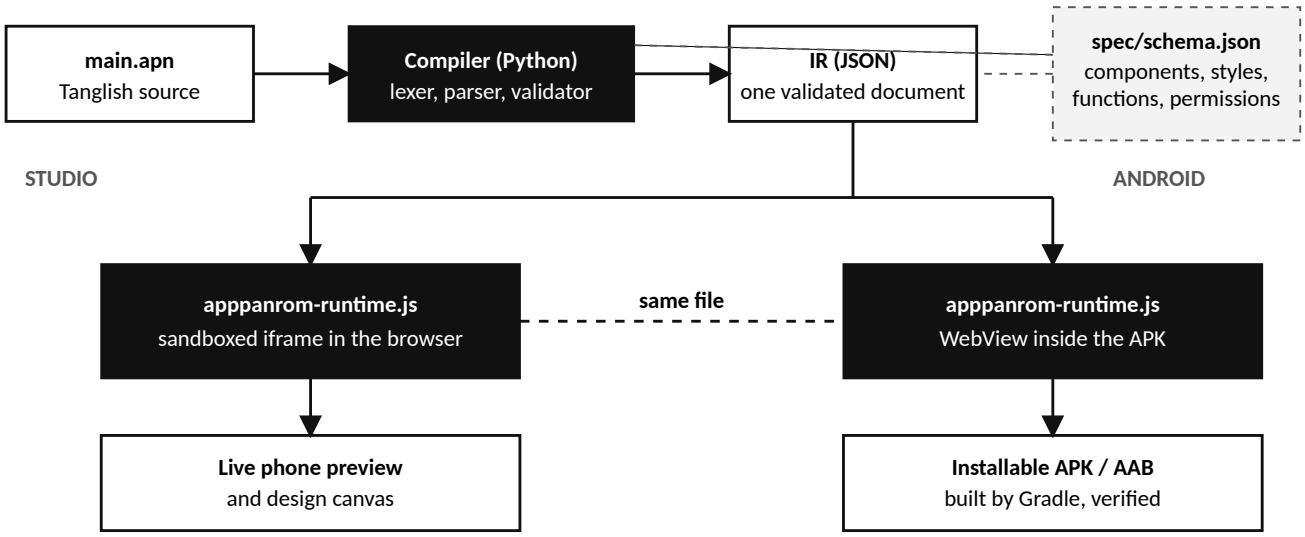


Figure 3.1: One compiler output, one runtime, two hosts.

> **WHY ONE RUNTIME**
>
> The most common failure of "preview" features is that the preview is a separate re-implementation of the real thing, so it slowly drifts. If a button is drawn differently in the preview than on the phone, the preview is worse than useless: it is misleading. AppPanrom avoids this by construction. There is exactly one file, `runtime/apppanrom-runtime.js`. The Studio embeds it in the preview iframe; the Android generator copies the same file into the APK's assets. Differences between preview and APK can only come from the host (browser versus Android WebView), never from the app logic.

3.2 Components and responsibilities

Component	Location	Responsibility
Language engine	`backend/apppanrom/lang/`	Lexer, parser, semantic validator and IR lowering; diagnostics; the visual-designer text-edit engine.
API server	`backend/apppanrom/main.py`	FastAPI application: projects, files, compile/preview, designer edits, builds, icon, system status.
Storage	`db.py`, `projects.py`	SQLite metadata; source files in one directory per project.
Android pipeline	`backend/apppanrom/android/`	Prerequisite detection, project generation, background Gradle runs, artifact verification, signing checks, icon validation.

Runtime	`runtime/apppanrom-runtime.js`	Interpreter and renderer shared by preview, design canvas and APK.
Spec	`spec/schema.json`	Single machine-readable definition of the language surface, read by the compiler, the runtime tests, the editor and the documentation.
Studio	`frontend/src/`	React single-page application: dashboard, workspace, editor, preview, designer, build panel.
Android template	`android-template/`	A minimal Gradle project (one Activity with a WebView) into which the generator injects per-app values.
CI	`.github/workflows/android-apk.yml`	Builds the example apps with the real Android SDK and publishes the APKs.

3.3 Data flow of a single keystroke

It helps to follow one interaction end to end. Suppose the learner types a character in the editor:

1. CodeMirror reports the change; the Workspace stores the text, marks the project "dirty" and starts two timers: autosave after 1,200 ms and validation after 450 ms.
2. When the validation timer fires, the Studio calls `POST /api/projects/{id}/preview` with the current buffer (not the saved file).
3. The backend calls `compile_source()`. The lexer produces tokens, the parser builds an AST, the compiler checks it and lowers it to IR. The response contains diagnostics and, when there are no errors, the IR.
4. The Studio shows diagnostics in the editor and keeps the latest successful IR for the design canvas.
5. When the learner presses Run, the same call is made and the IR is placed into a sandboxed iframe together with the runtime. The runtime renders the first screen.

The server never runs the app. It only compiles. Everything that executes a user's program runs in the browser (or later on the phone), inside the runtime's interpreter.

3.4 Repository layout

Repository tree

```
AppPanrom/
  backend/
    apppanrom/
      lang/        lexer.py parser.py compiler.py designer.py diagnostics.py spec.py
      android/     builder.py generate.py prereqs.py icon.py
      main.py      FastAPI app          projects.py  project store
      db.py        SQLite wrapper       config.py    environment settings
      cli.py       compile / android-project / make-keystore
    tests/         test_language.py test_api.py test_builds.py
                   test_designer.py test_signing_icon.py
  runtime/         apppanrom-runtime.js  test/runtime.test.mjs
  spec/            schema.json
  frontend/        src/ (React Studio)   e2e/run.mjs (browser end-to-end)
  android-template/ Gradle project with MainActivity.java
  examples/        *.apn sample programs
  docs/            language.md, report/, presentation/
  .github/workflows/android-apk.yml
```

3.5 Technology choices

Choice	Reason

Python for the compiler	Quick to write and test; the compiler runs only on the server, so there is no need for it to run on the phone.
Hand-written lexer and parser	The grammar is small and indentation-sensitive. A hand-written recursive-descent parser gives full control over error messages, which matters more to a learner than parsing speed.
JavaScript runtime	An Android WebView and a browser both run JavaScript, so one runtime can serve both. It also gives free, good-looking UI primitives (HTML and CSS).
Interpreter, not code generation	Turning user code into JavaScript and running it would require `eval` or generated script files, which is an injection risk. Interpreting a validated IR with a fixed function allow-list is safe by design.
FastAPI + SQLite	A local single-user tool needs no database server. SQLite gives durable project and build history in one file; FastAPI gives typed request validation and automatic API documentation.
React + CodeMirror 6	CodeMirror is small, fully local (works offline) and easy to integrate with custom languages. React keeps the many interacting panels manageable.
WebView app shell	The generated app is a thin native shell around the runtime. It builds with standard Gradle, needs no Kotlin, and keeps the APK small (about 0.5 MB).

4 The AppPanrom Language

4.1 Design philosophy

The language follows four rules that shape every decision below.

1. **Small.** A learner can hold the entire language in their head. There are 18 reserved words, 18 UI components and 47 built-in functions.
2. **Explicit.** There is no implicit conversion between text and numbers, no hidden global state, no operator overloading. `"Count: " + count` is an error that tells the learner to write `str(count)`.
3. **Friendly errors.** Every error carries a position and, where possible, a suggestion.
4. **Safe to run.** The language has no file, network, process or import capability. A program can only draw UI, keep its own variables and store small amounts of its own data.

4.2 Lexical structure

A program is a UTF-8 text file (at most 200,000 bytes). It is split into lines. Like Python, the language uses **indentation** to show blocks, but it is stricter: indentation must be exactly four spaces per level, and tabs are rejected.

Element	Rule
Comments	Start with `#` and run to the end of the line.
Names	Letters, digits and underscores, not starting with a digit.
Numbers	`42` or `3.14`. A leading `.5` is allowed.
Text	Single or double quotes. Escapes: `\n \t \" \' \\`. Curly "smart" quotes are reported with a hint because pasting from word processors often produces them.
Reserved words	`mari function return enna_na illana_enna illana ovvoru in irukkura_varai try catch and or not summa true false null`
Operators	`+ - * / %`, `== != < > <= >=`, `and or not`, indexing `[ ]`, calls `( )`.

Newlines inside brackets	Ignored, so a long call or list can be split over several lines.

4.3 Values and variables

There are six kinds of value: number, text, bool (`true` / `false`), list, dictionary and `null` . Variables are declared once with `mari` and changed with plain assignment:

```
Variables
mari count = 0              # declaration
count = count + 1           # assignment
mari names = ["Asha", "Ravi"]
names[0] = "Meena"          # list item assignment
mari user = {"name": "Kumar", "age": 21}
user["age"] = 22            # dictionary item assignment
```

Truthiness. In conditions, `null` , `false` , `0` , the empty text, the empty list and the empty dictionary count as false; everything else is true. **Equality** is structural: two lists with the same items are equal. **Ordering** (`<` and friends) works only between two numbers or two texts; anything else is a runtime error rather than a silent guess.

4.4 Control flow

Construct	Meaning
`enna_na cond:`	"If". Followed by an indented block.
`illana_enna cond:`	"Else if". Any number may follow an `enna_na` .
`illana:`	"Else".
`ovvoru x in items:`	"For each". Loops over a list, the characters of a text, or the keys of a dictionary. To repeat N times use `ovvoru i in range(N):` .
`irukkura_varai cond:`	"While". Repeats as long as the condition is true. A step limit stops endless loops.
`try: ... catch error:`	Runs the first block; if a runtime error occurs, the second block runs with the error message (as text) in `error` .
`summa`	"Nothing". A statement that does nothing; it keeps otherwise empty blocks valid.

4.5 Functions and handlers

Functions are defined at the top level with `function name(params):` and return values with `return` . A function used as an **event handler** must take no parameters, because the runtime calls handlers with no arguments. This is why helper functions in the example apps are separate from handler functions. Functions cannot be nested and cannot be passed around as values; there is no recursion limit other than the runtime's call-depth limit of 100.

4.6 Screens and components

A screen is declared with `screen_uruvakku("name"):` ("create screen") followed by an indented block of component calls. The first screen declared is the start screen unless `app_start_screen` says otherwise. Components are called like functions; containers take an indented block of children.

Component	Kind	Purpose and main options

`title`	leaf	Large heading.
`text_kaattu`	leaf	A paragraph of text ("kaattu" means "show").
`button`	leaf	`button(label, click_pannumbodhu=handler, enabled=true)`.
`input_uruvakku`	input	Single-line text field; options `label`, `placeholder`, `initial`; events `change_aagumbodhu`, `submit_pannumbodhu`.
`password_input`	input	Masked text field.
`number_input`	input	Numeric field; `input_value` returns a number or `null` when empty.
`checkbox_uruvakku`, `toggle_uruvakku`	input	True/false inputs (checkbox or switch).
`dropdown_uruvakku`	input	Choose one of `options=[...]`.
`image_kaattu`	leaf	Picture from an `https://` URL or `data:image/` URI.
`list_kaattu`	leaf	One row per item of a list.
`progress_kaattu`	leaf	Bar from 0 to 100; `text_color` style sets the fill colour.
`loading_kaattu`, `divider_kaattu`	leaf	Spinner; horizontal line.
`column_uruvakku`, `row_uruvakku`	container	Vertical and horizontal layout.
`card_uruvakku`	container	Raised container with border and padding.
`center_ama`	container	Centres its children on both axes.

Inputs are identified by name. Each input takes a fixed text name, for example `input_uruvakku("amount", ...)`, and the program reads it with `input_value("amount")` and writes it with `input_set("amount", value)`. The name must be a literal string so that the compiler can find every input without running the program.

Navigation

`screen_ku_poo("name")` ("go to screen") pushes a screen on a stack; `screen_back_poo()` pops it. On Android the system Back button pops the stack and exits only when it is empty. `screen_data_set` and `screen_data_get` pass small values between screens in memory.

4.7 Styling

Every component accepts `style={...}`, a dictionary literal of validated style keys. The style system is intentionally a small, checked subset of CSS. Each key belongs to a group, and each component allows only certain groups, so a key that makes no sense is an error ("Style 'font_size' does not apply to image_kaattu").

Type	Keys and allowed values
color (hex)	`background, text_color, border_color`: `"#2563EB"` or `"#FFF"`
number	`padding, margin, gap, corner_radius` (0-200), `border_width` (0-20), `font_size` (8-96), `opacity` (0-1)
size	`width, height`: a number, a percentage such as `"50%"`, or `"fill"`
enum	`font_weight` (normal, medium, bold); `text_align` (left, center, right); `align` (start, center, end, stretch); `justify` (start, center, end, space_between, space_around)
bool	`wrap, scroll, visible`

Values may be literals (checked at compile time) or expressions (checked again at runtime when the screen is drawn). That is how the example apps colour a progress bar green or red depending on state: `style={"text_color": budget_color()}`. Gradients, shadows and custom fonts are listed as unsupported and produce a clear message.

4.8 The standard library

All 47 built-in functions are listed in the appendix. They fall into groups: messages (`message_kaattu`, `toast_kaattu`, logging), conversion (`int float str bool type_of`), text (`text_length text_upper text_lower text_trim text_replace text_split text_join text_contains`), collections (`length append remove sort range dict_get dict_keys`), numbers (`min max sum round abs`), data (`json_encode json_decode`), time and randomness (`now_iso random_number random_choice`), inputs (`input_value input_set`), local storage (`data_save_pannu data_edu data_delete_pannu data_clear_pannu`), navigation, and device features (`clipboard_copy vibrate_pannu`).

Several names are Tanglish verbs that describe the action: `data_save_pannu` ("do save data"), `data_edu` ("take data"), `screen_ku_poo` ("go to the screen"), `vibrate_pannu` ("do vibrate"). Names that are plain English technical terms (`round`, `sort`, `str`) are kept as they are, because they are already familiar.

4.9 What is deliberately missing

The compiler keeps a list of names that are planned but not implemented: `api_get api_post api_put api_delete api_json dialog_kaattu app_icon_set app_config state_uruvakku state_get state_set ui_refresh screen_refresh vadivelu_mode enna_kodumai_debug summa_adirudhulla gradient_set shadow_set`. Using one produces "'api_get' is planned but not implemented yet" instead of the less helpful "Unknown function". There is also no `break` / `continue`, no modules or imports, and no first-class functions.

5 The Compiler

The compiler lives in `backend/apppanrom/lang/`. Its public entry point is `compile_source(source, defaults)`, which returns a result with a success flag, a list of diagnostics, and (on success) the IR. It runs four stages.

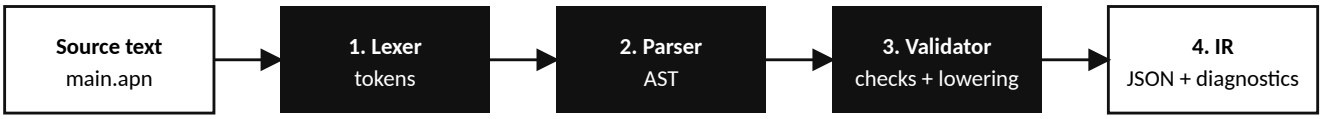


Figure 5.1: The four stages of compilation.

5.1 Stage 1: the lexer

The lexer (`lexer.py`, 137 lines) turns text into a flat list of tokens. Each token has a kind, a value and a line/column position. Beyond the usual NAME, KEYWORD, NUMBER, STRING and OP tokens, it emits three structural tokens that carry the indentation: **NEWLINE**, **INDENT** and **DEDENT**. This is the standard technique used by Python's own tokenizer, and it lets the parser treat blocks like braces.

backend/apppanrom/lang/lexer.py (lines 31-52) - Source size limit, tab check, indentation stack

```
31  def tokenize(source: str) -> list[Token]:
32      if len(source.encode("utf-8")) > MAX_SOURCE_BYTES:
33          raise SyntaxProblem(f"Source is larger than {MAX_SOURCE_BYTES} bytes.", 1, 1, "source_too_large")
34      tokens: list[Token] = []
35      indents = [0]
36      depth = 0  # open brackets: newlines are ignored inside them
37      lines = source.replace("\r\n", "\n").replace("\r", "\n").split("\n")
38      for lineno, raw in enumerate(lines, start=1):
39          i = 0
40          n = len(raw)
41          if depth == 0:
42              stripped = raw.lstrip(" ")
43              if "\t" in raw[: n - len(raw.lstrip())]:
44                  raise SyntaxProblem("Tabs are not allowed for indentation.", lineno, 1, "tab_indent",
45                                      "Use 4 spaces per indentation level.")
46              if stripped == "" or stripped.startswith("#"):
47                  continue
48              width = n - len(stripped)
49              if width % INDENT_WIDTH:
50                  raise SyntaxProblem(f"Indentation must be a multiple of {INDENT_WIDTH} spaces (found {widt
    h}).",
51                                      lineno, 1, "bad_indent", f"Use {INDENT_WIDTH} spaces per level.")
52              if width > indents[-1]:
```

The lexer keeps a stack of indentation widths. A deeper line pushes a new width and emits INDENT; a shallower line pops widths and emits one DEDENT per level. The width must be a multiple of four, and a line may only go one level deeper at a time. Blank lines and comment lines are skipped, and lines inside open brackets do not produce structure tokens, which is why a multi-line list literal works.

backend/apppanrom/lang/lexer.py (lines 74-92) - Scanning a line

```
74              elif ch in "\"'":
75                  value, i = _read_string(raw, i, lineno)
76                  tokens.append(Token("STRING", value, lineno, col, i + 1))
77              elif ch.isdigit() or (ch == "." and i + 1 < n and raw[i + 1].isdigit()):
78                  j = i
79                  seen_dot = False
80                  while j < n and (raw[j].isdigit() or (raw[j] == "." and not seen_dot)):
81                      seen_dot = seen_dot or raw[j] == "."
82                      j += 1
83                  text = raw[i:j]
84                  tokens.append(Token("NUMBER", float(text) if seen_dot else int(text), lineno, col, j + 1))
85                  i = j
86              elif ch.isalpha() or ch == "_":
87                  j = i
88                  while j < n and (raw[j].isalnum() or raw[j] == "_"):
89                      j += 1
90                  word = raw[i:j]
91                  tokens.append(Token("KEYWORD" if word in KEYWORDS else "NAME", word, lineno, col, j + 1))
92                  i = j
```

For the three-line program `mari clicks = 0 / function tap(): / clicks = clicks + 1` the real lexer output begins like this:

Token stream (abridged, from the real lexer)

```
KEYWORD 'mari'    line 1 col 1       KEYWORD 'function'  line 3 col 1
NAME 'clicks'     line 1 col 6       NAME 'tap'          line 3 col 10
OP '='            line 1 col 13      OP '('  OP ')'  OP ':'
NUMBER 0          line 1 col 15      NEWLINE
NEWLINE                              INDENT              line 4 col 1
                                     NAME 'clicks'  OP '='  NAME 'clicks'  OP '+'  NUMBER 1
                                     NEWLINE
                                     DEDENT              line 6 col 1
```

ERROR QUALITY

Because the lexer knows exact positions, its errors are precise: "Indentation must be a multiple of 4 spaces (found 6)", "String is not closed", "Curly (smart) quote found. Use straight quotes". Each is raised as a `SyntaxProblem` that carries the line, column, an error code and a suggestion. The first lexical or syntax error stops compilation, because after such an error the rest of the file cannot be trusted.

5.2 Stage 2: the parser

The parser (`parser.py`, 379 lines) is a **recursive-descent parser**: one method per grammar rule, each consuming tokens and returning a node. Nodes are plain Python dictionaries with a type tag `t` and a position `pos`, so they can be turned into JSON without any conversion. A nesting limit of 40 protects against pathological input.

Statements

backend/apppanrom/lang/parser.py (lines 112-136) - Statement dispatch

```
112     def _statement(self) -> dict:
113         t = self.cur
114         pos = self.pos(t)
115         if t.kind == "KEYWORD":
116             kw = t.value
117             if kw == "mari":
118                 self.advance()
119                 name = self.expect_name("a variable name after 'mari'")
120                 self.expect_op("=", "Declare variables like: mari count = 0")
121                 e = self.expr()
122                 self.end_of_statement()
123                 return {"t": "let", "n": name.value, "e": e, "pos": pos}
124             if kw == "summa":
125                 self.advance()
126                 self.end_of_statement()
127                 return {"t": "summa", "pos": pos}
128             if kw == "function":
129                 return self.function_def()
130             if kw == "return":
131                 self.advance()
132                 e = None
133                 if self.cur.kind not in ("NEWLINE", "EOF", "DEDENT"):
134                     e = self.expr()
135                 self.end_of_statement()
136                 return {"t": "return", "e": e, "pos": pos}
```

The method looks at the first token. A keyword selects the statement type (`mari`, `function`, `return`, `enna_na`, loops, `try`). Anything else is parsed as an expression, and what follows decides what it was: an `=` makes it an assignment (the left side must be a name or an index), a `:` makes it a component with a block of children, and otherwise it must be a call statement. A bare expression like `5 + 3` is rejected with "This statement does nothing".

Expressions and precedence

backend/apppanrom/lang/parser.py (lines 258-285) - Expression levels from lowest to highest precedence

```
258      def compare(self):
259          left = self.additive()
260          if self.cur.kind == "OP" and self.cur.value in COMPARE_OPS:
261              t = self.advance()
262              left = {"t": "bin", "op": t.value, "l": left, "r": self.additive(), "pos": self.pos(t)}
263              if self.cur.kind == "OP" and self.cur.value in COMPARE_OPS:
264                  self.err("Chained comparisons are not supported.", suggestion="Use 'and': a < b and b < c")
265          return left
266
267      def additive(self):
268          left = self.term()
269          while self.cur.kind == "OP" and self.cur.value in "+-":
270              t = self.advance()
271              left = {"t": "bin", "op": t.value, "l": left, "r": self.term(), "pos": self.pos(t)}
272          return left
273
274      def term(self):
275          left = self.unary()
276          while self.cur.kind == "OP" and self.cur.value in ("*", "/", "%"):
277              t = self.advance()
278              left = {"t": "bin", "op": t.value, "l": left, "r": self.unary(), "pos": self.pos(t)}
279          return left
280
281      def unary(self):
282          if self.is_op("-"):
283              t = self.advance()
284              return {"t": "un", "op": "-", "e": self.unary(), "pos": self.pos(t)}
285          return self.postfix()
```

Operator precedence is encoded in the call chain: `or` calls `and`, which calls `not`, which calls comparison, then addition, then multiplication, then unary minus, then postfix operators (indexing and calls), then atoms. Lower-precedence rules call higher-precedence rules, so `a + b * c` naturally groups as `a + (b * c)`. Chained comparisons such as `a < b < c` are rejected with the suggestion to use `and`, because their meaning differs between languages and would confuse learners.

Calls with keyword arguments

A call collects positional arguments and keyword arguments (`name=value`). A positional argument after a keyword argument and a repeated keyword are both errors. This is how `button("Tap", click_pannumbodhu=tap, style={...})` is read: one positional argument, then two keyword arguments.

The AST

The parser's result for the assignment `clicks = clicks + 1` on line 4 shows the shape: every node has a type, children and a position.

AST of "clicks = clicks + 1" (real output, abridged)

```
{"t": "assign",
 "target": {"t": "name", "n": "clicks", "pos": [4, 5]},
 "e": {"t": "bin", "op": "+",
       "l": {"t": "name", "n": "clicks", "pos": [4, 14]},
       "r": {"t": "num",  "v": 1,        "pos": [4, 23]},
       "pos": [4, 21]},
 "pos": [4, 5]}
```

Statements also record the last source line they cover (`end`) and where their header ends (`hend`). The compiler does not need these, but the visual designer does: they let it replace exactly the lines of one component without touching anything else (Chapter 7).

5.3 Stage 3: semantic validation and lowering

A syntactically correct program can still be wrong: it may call an undefined function, use a variable that was never declared or navigate to a screen that does not exist. The compiler (`compiler.py`, 524 lines) catches these, and unlike the parser it collects **all** semantic errors in one pass instead of stopping at the first. It also rewrites the AST into the IR.

backend/apppanrom/lang/compiler.py (lines 94-124) - The compile() method

```
94      def compile(self, source: str) -> CompileResult:
95          try:
96              program = parse(source)
97          except SyntaxProblem as e:
98              return CompileResult(False, [e.diagnostic])
99
100         for st in program:  # pass 1: declarations
101             t = st["t"]
102             if t == "function":
103                 if st["n"] in self.functions:
104                     self.error(st, f"Function '{st['n']}' is already defined.", "duplicate_function")
105                 elif st["n"] in self.spec["functions"] or st["n"] in self.spec["components"]:
106                     self.error(st, f"'{st['n']}' is a built-in name and cannot be redefined.", "reserved_nam
    e")
107                 else:
108                     self.functions[st["n"]] = st
109             elif t == "block_call" and st["f"] == "screen_uruvakku":
110                 name = self._screen_name(st)
111                 if name is not None:
112                     if name in self.screens:
113                         self.error(st, f"Screen '{name}' is declared more than once.", "duplicate_screen")
114                     else:
115                         self.screens[name] = st
116
117         ir_globals: list[dict] = []
118         ir_screens: dict[str, dict] = {}
119         for st in program:  # pass 2: bodies
120             t = st["t"]
121             if t == "call_stmt" and st["f"] in self.spec["config_functions"]:
122                 self._config(st)
123             elif t == "let":
124                 e = self._expr(st["e"], self.globals, None)
```

Compilation works in two passes over the top-level statements.

1. **Pass 1 (declarations).** Collect every function and every screen, so that a handler may be defined after the screen that uses it, and report duplicates and reserved names (a user function cannot be called `round` or `button`).
2. **Pass 2 (bodies).** Process configuration calls (`app_uruvakku` and friends), global `mari` declarations, then every function body and every screen body.

After both passes, cross-checks run: at least one screen must exist, every `screen_ku_poo("x")` target must exist (with a "Did you mean" hint), and every `input_value("x")` name must belong to a declared input component (a warning, not an error).

Scopes and the render-purity rule

Variables are checked against a scope: the function's parameters and local `mari` declarations plus the globals. Assigning to an undeclared variable is an error with a suggestion. There is one more rule that is central to the whole design:

backend/apppanrom/lang/compiler.py (lines 237-251) - Assignment checks

```
237         if t == "assign":
238             target = st["target"]
239             base = target
240             while base["t"] == "index":
241                 base = base["e"]
242             if base["t"] != "name":
243                 self.error(st, "Cannot assign to this expression.", "bad_assign")
244             else:
245                 if base["n"] not in scope and base["n"] not in self.globals:
246                     self.error(base, f"Variable '{base['n']}' is not declared.", "undefined_variable",
247                                self.suggest(base["n"], scope | self.globals) or f"Declare it first: mari {bas
    e['n']} = ...")
248                 elif ctx == "screen" and base["n"] not in scope:
249                     self.error(base, f"Screens cannot change global '{base['n']}' while drawing; change it in
    side a button handler function.",
250                                "render_mutation")
251             return {"t": "assign", "target": self._target(target, scope, ctx), "e": self._expr(st["e"], scop
    e, ctx), "pos": st["pos"]}
```

The branch `ctx == "screen"` forbids assigning to a global while a screen body is being drawn. Screens are meant to be a pure function of the state; changes to state belong in event handlers. The runtime enforces the same rule dynamically, as a second line of defence (Section 6.5).

Components

backend/apppanrom/lang/compiler.py (lines 315-331) - Validating a component call

```
315     def _component(self, st: dict, scope: set[str], spec: dict) -> dict:
316         name = st["f"]
317         self._note_use(name)
318         if st["t"] == "block_call" and not spec["container"]:
319             self.error(st, f"'{name}' cannot contain other components.", "no_block")
320         if st["t"] == "call_stmt" and spec["container"]:
321             self.error(st, f"'{name}' needs an indented block of children after ':'.", "needs_block",
322                        f"Write: {name}(...):  then indent the children.")
323         pos_spec = spec["positional"]
324         if len(st["args"]) > len(pos_spec):
325             self.error(st, f"{name} takes at most {len(pos_spec)} positional argument(s); use name=value for
    options.", "too_many_args", spec["doc"])
326         props: dict[str, dict] = {}
327         for i, a in enumerate(st["args"][: len(pos_spec)]):
328             props[pos_spec[i]["name"]] = self._typed(a, pos_spec[i]["type"], scope, f"{name} argument '{pos_s
    pec[i]['name']}'")
329         for p in pos_spec[len(st["args"]):]:
330             if p["required"] and p["name"] not in st["kw"]:
331                 self.error(st, f"{name} is missing required argument '{p['name']}'.", "missing_arg", spec["do
    c"])
```

For each component the compiler looks up its entry in `spec/schema.json` and checks: container components must have a block, leaf components must not; positional arguments and keyword options must exist and have the right literal type; event names must be allowed for that component and must name a function that exists and takes no parameters; style keys must be valid and applicable. Input components must have a literal name, which is recorded in a set of declared inputs.

Styles

backend/apppanrom/lang/compiler.py (lines 415-437) - Style validation

```
415      def _style_value(self, key: str, d: dict, v: dict):
416          lt = literal_type(v)
417          if lt is None:
418              return  # dynamic value: validated again by the runtime
419          typ = d["type"]
420          ok = True
421          if typ == "color":
422              ok = lt == "str" and bool(HEX_RE.fullmatch(v["v"]))
423              hint = 'Use a hex color like "#2563EB" or "#FFF".'
424          elif typ == "number":
425              ok = lt == "num" and d["min"] <= v["v"] <= d["max"]
426              hint = f"Use a number from {d['min']} to {d['max']}."
427          elif typ == "size":
428              ok = (lt == "num" and 0 <= v["v"] <= 2000) or (lt == "str" and bool(SIZE_RE.fullmatch(v["v"])))
429              hint = 'Use a number, a percentage like "100%", or "fill".'
430          elif typ == "enum":
431              ok = lt == "str" and v["v"] in d["values"]
432              hint = f"Allowed: {', '.join(d['values'])}."
433          else:  # bool
434              ok = lt == "bool"
435              hint = "Use true or false."
436          if not ok:
437              self.error(v, f"Invalid value for style '{key}'.", "bad_style_value", hint)
```

Literal style values are validated immediately against the style schema (colour pattern, numeric range, enum membership). Dynamic values, such as the result of a function call, cannot be checked statically, so the compiler keeps them in the IR and the runtime validates them when the screen is drawn.

Typing: what is and is not checked statically

The language is dynamically typed, so the compiler does not try to infer types. It only checks cases where both operands are literals (for example `"a" + 1`) or where a literal is passed to a parameter of a known type. Everything else is checked by the runtime with equally clear messages. Real compiler output for a small faulty program shows both kinds of help:

Source with mistakes

```
mari n = 1
screen_uruvakku("home"):
    text_kaatu("hi")
    button("Go", click_pannumbodhu=missing)
    text_kaattu("n: " + n)
```

Diagnostics produced (real output)

```
line 3, col 5   error  unknown_function    Unknown function 'text_kaatu'.
                                           Did you mean 'text_kaattu'?
line 4, col 36  error  undefined_handler   Handler function 'missing' is not defined.
                                           Define it: function missing():
```

Line 5 is not flagged: `n` is a variable, not a literal, so its type is not known at compile time. If the program runs, the runtime stops that screen with "Cannot use '+' between text and number. Use str(...) to convert numbers to text." and shows the position. The design trade-off is deliberate: keep the compiler simple and predictable, and make runtime errors just as friendly.

Suggestions

The "Did you mean" feature uses Python's `difflib.get_close_matches` with a similarity cut-off of 0.6 against the pool of known names (variables in scope, user functions, built-ins and component names). A typo in `text_kaattu` is therefore usually fixed by reading the message.

5.4 The shared specification

`spec/schema.json` is the single description of the language surface. It lists themes, configuration functions, style groups and style definitions, unsupported styles, every component (with positional arguments, options, events, allowed style groups and whether it is a container), every built-in function (with minimum and maximum argument counts), the names reserved for future work, and the Android permission map. It is read by:

- the **compiler**, to validate calls and styles;
- the **runtime tests**, which assert that the runtime implements exactly the functions and components in the spec (a test fails if either side drifts);
- the **Studio**, which fetches it from `GET /api/language/spec` to highlight built-in names, populate the palette and inspector, and render the in-app language reference;
- the **designer**, to know which properties and styles each component accepts.

> **WHY A SHARED SPEC**
>
> Without it, the same facts would be written down in four places: the compiler, the runtime, the editor and the documentation. Whenever a component was added, one of them would be forgotten. With one JSON file as the source of truth and tests that compare it with the runtime, adding a function is a one-place change and forgetting the implementation fails the build.

5.5 Stage 4: the intermediate representation

If there are no errors, the compiler produces the IR, a JSON object that is the only thing the renderers ever see:

IR top-level keys

```
{ "ir_version": 1,
  "app":          {"name", "id", "version", "theme", "start_screen"},
  "style_schema": { ...copied from the spec so the runtime can validate dynamic styles },
  "permissions":  [ {"permission", "reason", "used_by": [...]} ],
  "globals":      [ {"t": "let", "n": ..., "e": <expression>} ],
  "functions":    { name: {"params": [...], "body": [...statements]} },
  "screens":      { name: {"style": ..., "body": [...statements]} },
  "screen_order": [ "home", ... ] }
```

For the three-line example program, the real IR for the button is:

A component in the IR (real output)

```
{"t": "comp", "c": "button",
 "props":  {"label": {"t": "str", "v": "Tap", "pos": [7, 12]}},
 "events": {"click_pannumbodhu": "tap"},
 "style":  null,
 "pos":    [7, 5]}
```

Two details are worth noting. First, **expressions stay as trees**: the IR does not contain precomputed values, because the value of `"Count: " + str(count)` depends on state and must be evaluated each time the screen is drawn. Second, **events refer to functions by name**, not by code. An event can therefore only ever call a function that the compiler verified to exist. The `permissions` list is derived automatically from the features used, for example `image_kaattu` adds `INTERNET` and `vibrate_pannu` adds `VIBRATE`, so the Android manifest declares exactly what the program needs and nothing more.

5.6 Diagnostics

A `Diagnostic` has a severity (error or warning), a code, a message, line and column, an optional end column and an optional suggestion. The API returns them as JSON; the editor turns them into underlines (using the end column or, if missing, the length of the word at that position) and the Diagnostics tab lists them with a jump-to-line action. Only errors block a run or a build; warnings (such as an undeclared input name) do not.

6 The Runtime

The runtime (`runtime/apppanrom-runtime.js` , 1,205 lines, no dependencies) is the heart of the product. It runs in three places: in the Studio's preview iframe, in the Studio's design canvas, and inside every generated APK. It is also loadable in Node.js, which is how the 36 runtime tests run without a browser.

It exposes two layers:

- `createRuntime(ir, host)` : the **headless runtime**. An interpreter plus a function that builds a *view tree* (plain objects describing what to draw). It touches no DOM.
- `mount(ir, rootElement, host)` : the **DOM renderer**. It builds on the headless runtime, turns the view tree into HTML elements, wires events and handles toasts, dialogs and design mode.

The `host` object is how the environment plugs in: where to send log lines, how to show toasts and messages, what storage to use, and (for tests) a clock. This keeps the interpreter independent of its surroundings.

6.1 The execution model

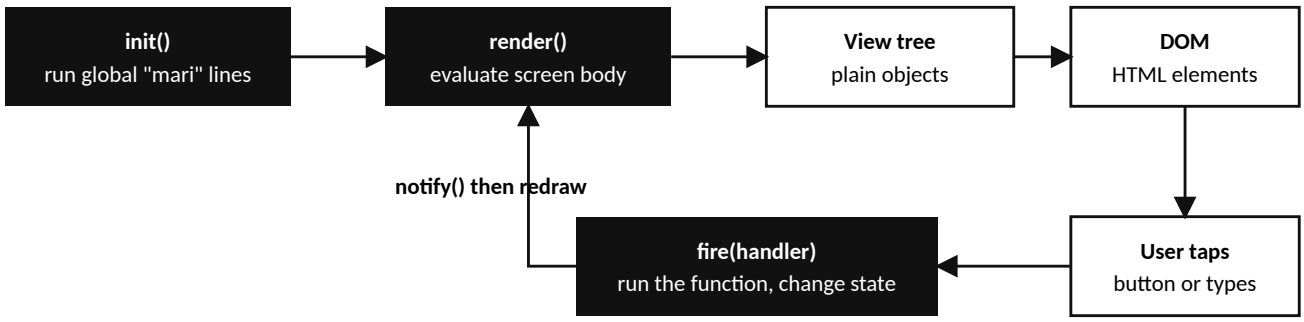


Figure 6.1: The runtime cycle. State changes only in handlers; every change redraws the screen from the new state.

1. **init.** Run the program's top-level `mari` lines in order to create the global variables. The navigation stack starts with the start screen.
2. **render.** Evaluate the current screen's body. Component statements append entries to a view tree; ordinary statements such as `enna_na` and `ovvoru` run normally, so conditional UI and generated lists work.
3. **draw.** The DOM renderer turns the view tree into elements.
4. **events.** A tap calls `fire(handlerName)` , which runs that function (it may change global variables, show a toast, navigate or save data) and then notifies the renderer, which redraws.

This is a deliberately simple "immediate mode" model, close to how React renders from state. Its great advantage for learners is that there is nothing to remember about updating the screen: change a variable and the screen follows.

6.2 Values and safety helpers

Lists are JavaScript arrays, text and numbers are primitives, and dictionaries are created with `Object.create(null)` so that a user key named `__proto__` or `constructor` cannot touch JavaScript's prototype chain (there is a dedicated test for dictionary keys named like JavaScript built-ins). Conversion to text (`toText`), truthiness and deep equality are implemented once and used by every operator and function.

6.3 Limits and errors

runtime/apppanrom-runtime.js (lines 19-28) - Resource limits

```
  var LIMITS = {
    steps: 1000000, // interpreter steps per entry point (init / event / render)
    depth: 100, // user-function call depth
    listItems: 100000,
    stringLength: 1000000,
    logLength: 2000,
    storageValue: 100000, // chars per stored value
    storageTotal: 1000000, // chars per app
    keyLength: 100,
  };
```

runtime/apppanrom-runtime.js (lines 234-247) - Step counting and the guard

```
    function tick(pos) {
      if (++steps > LIMITS.steps) throw new ApError('Program ran too long (more than ' + LIMITS.steps + ' steps); it may contain an endless loop.', pos, true);
    }
    function guarded(fn) {
      steps = 0; depth = 0;
      try { return fn(); } catch (e) {
        if (e && e.isApError) throw e;
        if (e instanceof RangeError) throw new ApError('Too much recursion.', null, true);
        throw e;
      }
    }
    function formatError(e) {
      return 'Runtime error' + (e.pos ? ' (line ' + e.pos[0] + ', col ' + e.pos[1] + ')' : '') + ': ' + e.message;
    }
```

Every evaluated expression and statement calls `tick()`, which counts interpreter steps. After 1,000,000 steps in one entry point (startup, one event, or one screen draw), the run stops with "Program ran too long; it may contain an endless loop." The call depth is capped at 100, lists at 100,000 items and texts at 1,000,000 characters. A JavaScript `RangeError` (stack overflow) is converted to "Too much recursion". These are **fatal** errors that `try / catch` cannot catch, so a program cannot hide an endless loop behind `try`.

Ordinary runtime errors (wrong type, index out of range, division by zero, missing dictionary key) are `ApError` objects with a position. In an event handler they are logged and shown as a toast ("Error: …") and never crash the app. In a screen draw, the screen is replaced by an error box showing the message and position, so a mistake is visible and located instead of silent.

6.4 Evaluating the IR

runtime/apppanrom-runtime.js (lines 540-557) - Calling functions and executing statements

```
    function invoke(name, args, pos) {
      var fn = ir.functions[name];
      if (fn) return callUser(name, fn, args, pos);
      var s = STD[name];
      if (!s) throw new ApError("Unknown function '" + name + "'.", pos);
      if (s.effect && rendering) throw new ApError(name + ' cannot be used while the screen is being drawn; call it from a button/event handler function.', pos);
      return s.fn(args);
    }
    function callUser(name, fn, args, pos) {
      if (args.length !== fn.params.length) throw new ApError("Function '" + name + "' expects " + fn.params.length + ' argument(s).', pos);
      if (++depth > LIMITS.depth) throw new ApError('Too much recursion (more than ' + LIMITS.depth + ' nested calls).', pos, true);
      var env = new Map();
      fn.params.forEach(function (p, i) { env.set(p, args[i]); });
      try {
        var r = execBlock(fn.body, env, null);
        return r ? r.v : null;
      } finally { depth--; }
    }
```

`invoke(name, args)` first looks for a user function in the IR, then for a built-in in the table `STD`. There is no third option: a name that is neither does not exist, so a program can never reach host objects such as `window` or `fetch`. User functions run in a fresh environment (a `Map` of local variables) so that locals never leak. `execInner` is a `switch` over the statement types: `let`, `assign`, `if`, `for`, `while`, `return`, `try`, `summa`, `call_stmt`, `screen_title` and `comp`.

Return values travel as a small object `{v: value}` returned from `execBlock`; a non-null result means "a return happened", which unwinds nested blocks without using JavaScript exceptions for normal control flow.

The standard library table

Each built-in is registered with `def(name, fn, effect)`. The third argument marks **effect functions**, those that change the outside world (messages, toasts, saving data, navigation, vibration, setting inputs). They are refused while a screen is being drawn, which leads to the key design rule of the next section. Built-ins validate their own arguments with a helper `need()` that produces messages such as "round argument 1 must be a number, not text".

6.5 Pure rendering

runtime/apppanrom-runtime.js (lines 714-730) - render() and fire()

```
714     rt.render = function () {
715       var name = stack[stack.length - 1], sc = ir.screens[name];
716       return guarded(function () {
717         rendering = true;
718         try {
719           var frame = { kids: [], title: null }, env = new Map();
720           execBlock(sc.body, env, frame);
721           var style = {};
722           if (sc.style) {
723             var res = validateStyle(evalExpr(sc.style, env), styleSchema);
724             style = res.valid;
725             res.problems.forEach(function (p) { warnOnce('Style ignored on screen ' + name + ': ' + p + '.');
    });
726           }
727           return { screen: name, title: frame.title === null ? ir.app.name : frame.title, style: style, child
    ren: frame.kids };
728         } finally { rendering = false; }
729       });
730     };
```

`render()` sets a `rendering` flag. While it is true, effect functions and assignments to globals throw an error that explains the rule ("toast_kaattu cannot be used while the screen is being drawn; call it from a button/event handler function"). Why? If drawing could change state, every redraw could change what the next redraw produces, leading to endless loops and unpredictable screens. Making the screen a pure function of the state is what allows the runtime to redraw freely after every event, and what makes the designer able to re-render at any moment without side effects. The compiler checks the same rule statically where it can, and the runtime enforces it for everything the compiler cannot see.

6.6 Building components

runtime/apppanrom-runtime.js (lines 641-656) - buildComponent

```
641     function buildComponent(n, env) {
642       var props = {};
643       Object.keys(n.props).forEach(function (k) { props[k] = evalExpr(n.props[k], env); });
644       var view = { c: n.c, props: normalizeProps(n.c, props, n.pos), events: n.events || {}, style: {}, pos:
    n.pos };
645       if (n.style) {
646         var raw = evalExpr(n.style, env), res = validateStyle(raw, styleSchema);
647         view.style = res.valid;
648         res.problems.forEach(function (p) { warnOnce('Style ignored on ' + n.c + ' (line ' + n.pos[0] + '): '
    + p + '.'); });
649       }
650       if (n.children) {
651         var inner = { kids: [], title: null };
652         execBlock(n.children, env, inner);
653         view.children = inner.kids;
654       }
655       return view;
656     }
```

`buildComponent` evaluates the component's property expressions in the current environment, normalises them into a typed set of properties (for example, a progress value is clamped to the range 0-100, a dropdown must have at least one option, an image source must be `https://` or `data:image/`), validates the style dictionary against the schema, and, for containers, runs the child block to collect the children. Invalid style values are dropped and reported once as a console warning with the line number, so one bad colour never blanks a screen.

Inputs

Input values live in a separate store, keyed by the input's name, not inside the view tree. The view tree is rebuilt on every draw, but the text the user typed must survive. The runtime scans the IR once at startup (`scanInputs`) to learn every input's kind and initial value, so that `input_value("name")` works even before that input has been drawn, for example in a handler of another screen. Number inputs return a number or `null`; checkboxes and toggles return booleans.

6.7 Local storage

runtime/apppanrom-runtime.js (lines 410-418) - data_save_pannu / data_edu / delete / clear

```
410     def('data_save_pannu', function (a) {
411       var k = skey(a[0]), s;
412       try { s = JSON.stringify(a[1]); } catch (e) { throw new ApError('data_save_pannu: value cannot be saved
    (' + e.message + ').'); }
413       if (s === undefined || s.length > LIMITS.storageValue) throw new ApError('data_save_pannu: value is too
    large (max ' + LIMITS.storageValue + ' characters).');
414       var total = ownKeys().reduce(function (n, key) { return key === k ? n : n + (storage.getItem(key) ||
    '').length; }, 0) + s.length;
415       if (total > LIMITS.storageTotal) throw new ApError('Storage is full (max ' + LIMITS.storageTotal + ' ch
    aracters per app).');
416       try { storage.setItem(k, s); } catch (e) { throw new ApError('Storage is full or unavailable: ' + e.mes
    sage); }
417       return null;
418     }, true);
```

Persistent data is stored as JSON text under keys prefixed with `ap:<namespace>:`. In the APK the namespace is the application id and the backend is the WebView's `localStorage`; in the preview it is the project and the data is bridged through `postMessage` (Chapter 9); in tests it is an in-memory object. The prefix means one app cannot read another's data. Limits are enforced on every save: key up to 100 characters, value up to 100,000 characters, total up to 1,000,000 characters per app. The documentation states plainly that this storage is not encrypted and must not hold passwords or tokens.

6.8 The DOM renderer

The renderer injects a small stylesheet (about 50 rules) with CSS variables for the light and dark themes (`--bg` , `--fg` , `--surface` , `--border` , `--accent` , `--muted`). Components map to semantic HTML: `title` becomes an `h1` , buttons are real `button` elements, inputs get `label` elements connected by `for` , toggles are checkboxes with `role="switch"` , progress bars are `progress` elements with ARIA attributes, dialogs have `role="alertdialog"` . Using standard elements gives keyboard and screen-reader support for free.

Because the whole screen is rebuilt after each event, the renderer takes care to preserve what a user would notice losing: the focused input and its cursor selection, and the scroll position (if the screen did not change). During IME composition (typing Tamil or Chinese text with an input method) it defers redraws so that half-composed characters are not destroyed.

Style application

`applyStyle` translates validated style keys into inline CSS: `padding` to `padding` , `align` and `justify` to `align-items` and `justify-content` , `width: "fill"` to `100%` , `height: "fill"` to `flex: 1` , and so on. Because only allow-listed keys with validated values reach this function, no user text is ever interpreted as CSS.

7 The Visual Designer

The designer lets a learner build screens by dragging and dropping, much like the layout editor of Android Studio, with one important difference: **there is no separate design file**. A conventional designer keeps its own model (an XML layout, a canvas state) and tries to keep it in sync with the code. Such synchronisation is notoriously fragile. AppPanrom's designer has no model of its own: `main.apn` is the only source of truth, and each designer action is turned into a small **text edit** of that file.

7.1 The two-way loop

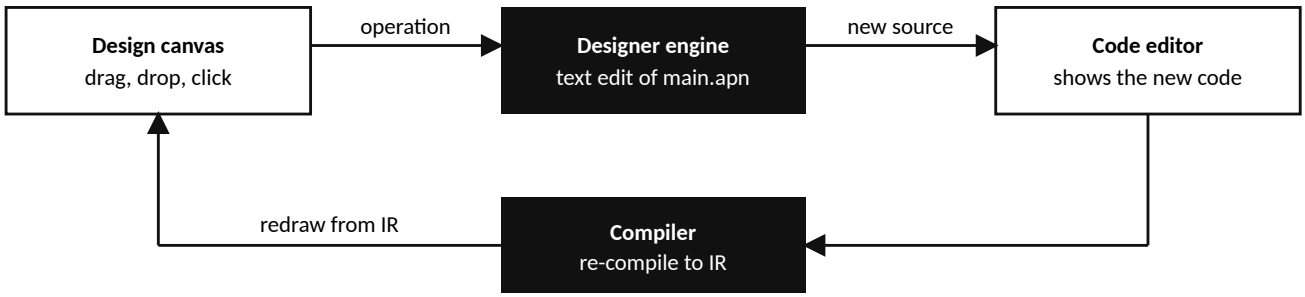


Figure 7.1: Designer actions rewrite the code; any code change recompiles and redraws the canvas.

Typing in the editor follows the same path in reverse: the code changes, the compiler produces new IR, and the canvas is updated in place (keeping the current screen, selection and scroll position). Both directions pass through the compiler, so the design view can never show something the code does not say. If the code has errors, the designer freezes on the last valid design and shows a banner instead of guessing.

7.2 Addressing components by source line

How does a click on the canvas identify "which line of code is this"? The runtime, when rendering in design mode, stamps every element with `data-ap-line` , the line number recorded in the IR at compile time. A component is therefore addressed by the line on which its statement starts. On the server, `locate(program, line)` walks the AST

to find the statement that begins on that line and remembers its parent list, so edits know their siblings.

backend/apppanrom/lang/designer.py (lines 145-157) - Locating a statement by line

```
145 def locate(program: list[dict], line: int) -> Loc | None:
146     def walk(stmts, parent, ancestors):
147         for n in stmts:
148             if n["pos"][0] == line:
149                 return Loc(n, stmts, parent, ancestors)
150             if n["pos"][0] <= line <= n["end"]:
151                 for sub in _child_lists(n):
152                     found = walk(sub, n, ancestors + [n])
153                     if found:
154                         return found
155         return None
156 
157     return walk(program, None, [])
```

7.3 Operations

Operation	What it does to the text
`insert`	Adds a snippet for a new component before, after or inside a target (or at the end of the screen).
`move`	Cuts a component's lines, re-indents them to the destination level and pastes them there.
`delete`	Removes the component's lines; leaves `summa` if its container would become empty.
`duplicate`	Copies the lines below the original; input names are made unique.
`set_prop`	Regenerates the component's header with a changed property.
`set_style`	Adds, changes or removes one key of the component's `style` dictionary.
`set_event`	Connects or disconnects an event handler by function name.
`add_screen`, `create_handler`	Appends a new screen, or inserts a new handler function above the first screen.

Line edits applied bottom-up

A move is both a removal and an insertion. If the lines were edited top to bottom, the first edit would shift every later line number and corrupt the second. The engine therefore collects all edits in **original** coordinates and applies them from the bottom of the file upwards, so earlier edits never disturb positions that are still to be applied.

backend/apppanrom/lang/designer.py (lines 186-204) - The Edits class

```
186 class Edits:
187     def __init__(self):
188         self.items: list[tuple[str, int, int, list[str]]] = []  # (kind, start, end, lines); 0-based, end exc
    lusive
189 
190     def insert(self, index: int, lines: list[str]):
191         self.items.append(("insert", index, index, lines))
192 
193     def remove(self, start: int, end: int):
194         self.items.append(("remove", start, end, []))
195 
196     def apply(self, lines: list[str]) -> list[str]:
197         out = list(lines)
198         # descending by start; at equal start, removals first so inserted lines land where the removed ones w
    ere
199         for kind, a, b, new in sorted(self.items, key=lambda it: (it[1], 0 if it[0] == "insert" else 1), reve
    rse=True):
200             if kind == "remove":
201                 del out[a:b]
202             else:
203                 out[a:a] = new
204         return out
```

A BUG FIXED DURING DEVELOPMENT

At equal positions, an insertion and a removal conflict: removing the placeholder `summa` and inserting a new component at the same spot must put the new line where the old one was, not delete it. The ordering rule (descending position, and at the same position the removal is applied first) handles exactly this case. The randomised tests in the suite (150 random operations in a row) exist to keep this class of problem from returning.

The insertion point

backend/apppanrom/lang/designer.py (lines 237-257) - Where does a dropped component go?

```
237 def _insertion_point(program, target: Loc | None, screen: Loc | None, position: str, edits: Edits) -> tuple[i
    nt, int]:
238     """Returns (0-based index to insert at, indent). May schedule removal of a lone `summa` placeholder."""
239     if position == "inside":
240         box = target or screen
241         if box is None or not is_container(box.node):
242             raise DesignError("You can only drop inside a container or a screen.", "not_a_container")
243         kids = box.node["children"]
244         indent = _indent_of(kids[0]) if kids else _indent_of(box.node) + INDENT
245         if len(kids) == 1 and kids[0]["t"] == "summa":
246             edits.remove(kids[0]["pos"][0] - 1, kids[0]["end"])
247             return kids[0]["pos"][0] - 1, indent
248         return box.node["end"], indent
249     if target is None:
250         raise DesignError("A target component is required.", "bad_target")
251     if target.parent is None or not _in_screen(target) or is_screen(target.node):
252         raise DesignError("You can only drop next to components inside a screen.", "bad_target")
253     if position == "before":
254         return target.node["pos"][0] - 1, _indent_of(target.node)
255     if position == "after":
256         return target.node["end"], _indent_of(target.node)
257     raise DesignError("Position must be before, after or inside.", "bad_position")
```

The three positions map directly onto the dashed and solid guides shown on the canvas while dragging: **inside** a container (dashed outline), **before** or **after** a sibling (a blue line). For "inside" the new line goes at the end of the container's block with the same indentation as its existing children; a lone `summa` placeholder is removed in the same edit.

Regenerating headers

For property, style and event edits the designer does not patch text. It modifies the parsed call node and writes the header back with `unparse()`, a small function that converts an expression tree to text with the minimum parentheses needed by operator precedence. Long headers are wrapped over several lines. The component's body and everything outside it are not touched.

backend/apppanrom/lang/designer.py (lines 480-496) - Setting one style key

```
480 def _op_set_style(program, lines, source, op):
481     loc = _comp_for_edit(program, op)
482     node, key = loc.node, op.get("key")
483     spec = load_spec()
484     if key not in spec["styles"]:
485         raise DesignError(f"Unknown style '{key}'.", "unknown_style")
486     allowed = {k for g in spec["components"][node["f"]]["style_groups"] for k in spec["style_groups"][g]}
487     if key not in allowed:
488         raise DesignError(f"Style '{key}' does not apply to {node['f']}.", "style_not_applicable")
489     style = node["kw"].get("style")
490     if style is None or style["t"] != "dict":
491         if style is not None:
492             raise DesignError("This component's style is not a plain {...} dictionary; edit it in the code.",
    "dynamic_style")
493         style = {"t": "dict", "items": [], "pos": [0, 0]}
494     items = list(style["items"])
495     idx = next((i for i, (k, _) in enumerate(items) if k["t"] == "str" and k["v"] == key), None)
496     val = _value_node(op.get("value"))
```

7.4 Never write invalid code

backend/apppanrom/lang/designer.py (lines 360-367) - apply_op: the safety net

```
360      before = len(_errors(source)) if kind in ("set_prop", "set_style", "set_event") else 0
361      new_lines, select = fn(program, lines, source, op)
362      new_source = _check(_join(new_lines))
363      if kind in ("set_prop", "set_style", "set_event"):
364          after = _errors(new_source)
365          if len(after) > before:
366              raise DesignError(after[0].message + (f" {after[0].suggestion}" if after[0].suggestion else ""),
     "invalid_value")
367      return {"source": new_source, "line": select}
```

After computing new text, `_check()` parses it. If it no longer parses, the operation is rejected and the file is unchanged. For property, style and event edits the compiler also runs before and after, and if the edit added an error (for example `font_size: 4`, below the minimum of 8), the edit is rejected with the compiler's own message, which the inspector shows next to the field. The result is a guarantee: **a designer action either produces valid code or does nothing.** A randomised test applies 150 operations in sequence and asserts that the file still compiles afterwards.

7.5 A real edit

Starting from this source, the learner drops a Button after the title, then sets its background colour in the inspector.

Before

```
app_uruvakku("D")
screen_uruvakku("home"):
    column_uruvakku(style={"gap": 8}):
        title("Hello")
```

After op = insert(button, after line 4) (real output)

```
app_uruvakku("D")
screen_uruvakku("home"):
    column_uruvakku(style={"gap": 8}):
        title("Hello")
        button("Button")                        <- inserted at the same indentation, line 5 selected
```

After op = set_style(line 5, background = "#2563EB") (real output)

```
        button("Button", style={"background": "#2563EB"})   <- header regenerated, nothing else changed
```

7.6 The inspector and the canvas

Selecting a component calls `POST /design/node`, which returns a description: its properties (each with both a literal value and its source expression), events, style values, and which properties, events and styles are allowed (from the spec). The inspector builds typed controls from that: text boxes, number boxes, a colour picker with hex entry, dropdowns for enums, true/false selectors. A button marked ***ƒx*** switches a field to a code expression such as `"Count: " + str(n)`. Choosing "New handler" writes a `function` into the code and connects the event to it as one visible edit.

The canvas is the real runtime mounted in **design mode**: clicks select instead of running handlers, inputs are inert, elements are draggable, and drag-over computes a drop zone from the pointer position (the first quarter of a container's height means "before", the last quarter "after", the middle "inside"). The runtime reports a drop with its type, target line and position, and the Studio forwards that to the designer engine. The canvas is mounted inside a **shadow root** so the Studio's own styles cannot leak into the phone and vice versa.

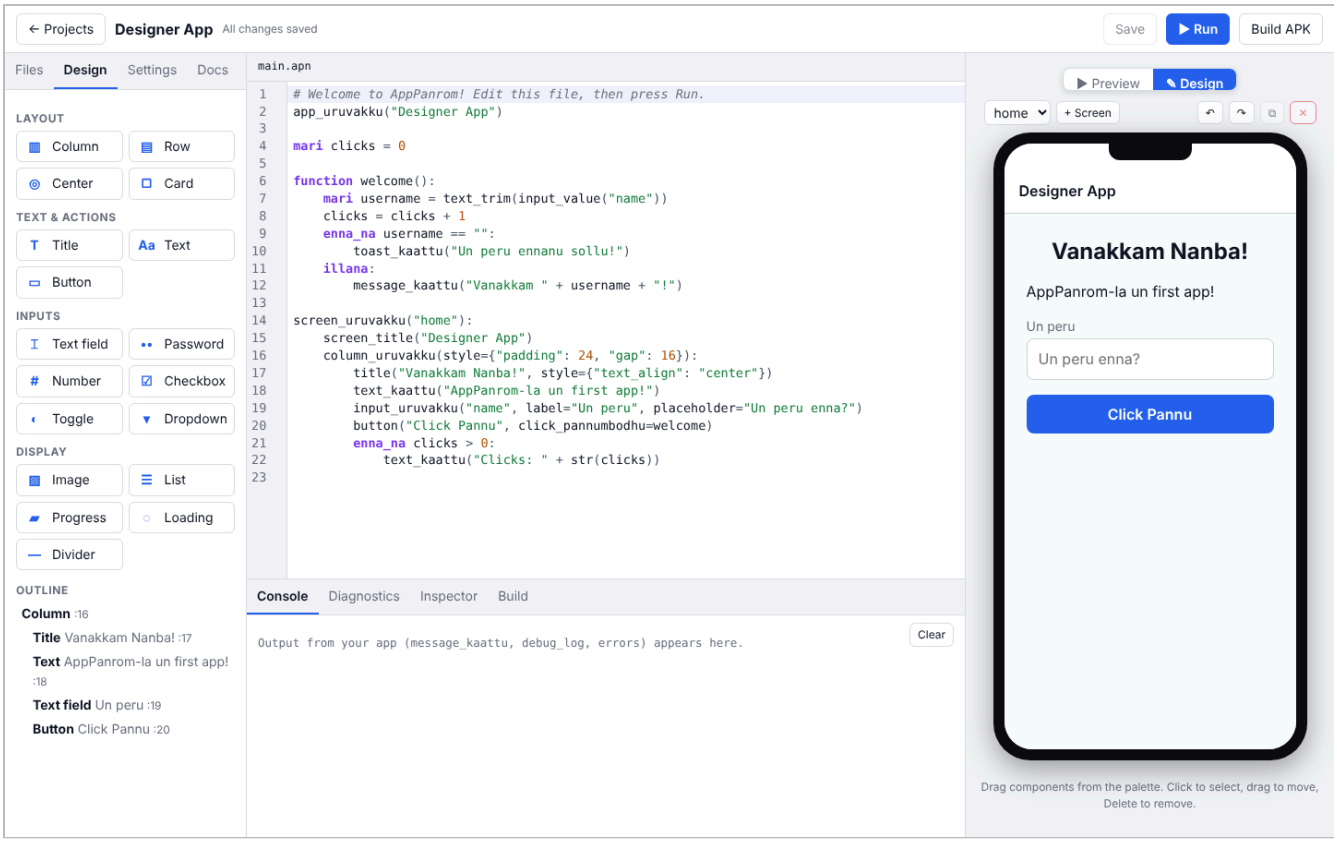


Figure 7.2: The visual designer. Palette and outline on the left, code in the centre, phone canvas on the right.

8 The Backend

The backend is a FastAPI application created by the factory `create_app(settings)` in `main.py`. Using a factory means the tests can build an application with temporary directories and fake settings, and the production server can build it from environment variables. The application owns three collaborators: a `Database`, a `ProjectStore` and a `BuildManager`.

8.1 The HTTP API

Endpoint	Purpose
`GET /health`	Liveness check and version.
`GET /api/system/status`	Android prerequisites, signing status and limits.
`GET /api/language/spec`	The shared language specification.
`GET, POST /api/projects`	List and create projects.
`GET, PATCH, DELETE /api/projects/{id}`	Read, rename or change metadata, delete (including builds).
`GET /api/projects/{id}/files` `GET, PUT, DELETE .../files/{name}`	List, read, save and delete project files.
`POST .../validate`, `POST .../preview`	Compile the given buffer (or the saved file); return diagnostics, and for preview also the IR.

`POST .../design/edit`, `POST .../design/node`	Apply one designer operation; describe the component at a line.
`PUT, GET, DELETE .../icon`	Upload, fetch and remove the app icon (PNG).
`POST, GET .../builds`	Start a build (`debug_apk` , `release_apk` , `release_aab`) and list builds.
`GET /api/builds/{id}`, `.../logs?offset=`, `.../artifact`	Build status, log streaming by byte offset, download of a verified artifact.

FastAPI also serves interactive documentation at `/docs`. If the Studio has been built (`frontend/dist` exists), the same server mounts it at `/`, so a single process serves the whole product at one address.

Statelessness of compile and design endpoints

backend/apppanrom/main.py (lines 217-232) - compile, preview and design endpoints

```
217     @app.post("/api/projects/{project_id}/preview")
218     def preview(project_id: str, body: SourceBody | None = None):
219         """Compiles to IR. The Studio renders the IR with the shared runtime; nothing is executed server-sid
    e."""
220         r = compile_project(project_id, body.source if body else None)
221         return r.to_dict()
222 
223     # ---- visual designer: stateless text edits of the editor buffer (the source file stays the single sourc
    e of truth)
224     @app.post("/api/projects/{project_id}/design/edit")
225     def design_edit(project_id: str, body: DesignEdit):
226         store.get(project_id)
227         return apply_op(body.source, body.op)
228 
229     @app.post("/api/projects/{project_id}/design/node")
230     def design_node(project_id: str, body: DesignNode):
231         store.get(project_id)
232         return describe(body.source, body.line)
```

The compile and design endpoints take the **text** as input. They do not read or write the project's file. This is what allows the Studio to validate the unsaved editor buffer, and to let the designer edit the buffer first and have the normal autosave persist it afterwards. It also means these endpoints have no side effects, so they are trivially safe to call repeatedly.

8.2 Uniform errors

Every error returned by the API has the same shape, `{"error": {"code", "message", "details?"}}`. Separate handlers convert application errors (not found, invalid input, designer errors), request-validation failures and unexpected exceptions into that shape. The last handler returns a generic "Internal server error" so that stack traces, file paths and library details never reach a client. Request bodies larger than 1,000,000 bytes are rejected by a middleware before they are read.

8.3 Projects and files

Project metadata is a row in SQLite; the source is a file in a directory named after the project id. The store (`projects.py` , 210 lines) is careful about everything that comes from the user:

backend/apppanrom/projects.py (lines 15-19) - Identifier and file-name patterns

```
15 PROJECT_ID_RE = re.compile(r"^p_[0-9a-f]{12}$")
16 FILE_NAME_RE = re.compile(r"^[A-Za-z0-9_][A-Za-z0-9_\-]{0,63}\.(apn|md|txt)$")
17 MAIN_FILE = "main.apn"
18 ICON_FILE = "icon.png"
19 THEMES = ("light", "dark")
```

backend/apppanrom/projects.py (lines 163-174) - Safe file access and atomic writes

```
163     def write_file(self, project_id: str, name: str, content: str) -> dict:
164         proj = self.get(project_id)
165         p = self._file(project_id, name)
166         if len(content.encode("utf-8")) > self.s.max_file_bytes:
167             raise Invalid(f"File is larger than {self.s.max_file_bytes} bytes.", "file_too_large")
168         if not p.exists() and len(proj["files"]) >= self.s.max_files_per_project:
169             raise Invalid(f"A project can have at most {self.s.max_files_per_project} files.", "limit")
170         tmp = p.with_suffix(p.suffix + ".tmp")
171         tmp.write_text(content, encoding="utf-8")
172         tmp.replace(p)
173         self.db.execute("UPDATE projects SET updated_at=? WHERE id=?", (now(), project_id))
174         return {"name": name, "size": len(content.encode("utf-8"))}
```

- Project ids are generated (`p_` plus 12 hex digits) and matched with `fullmatch`. A request with any other id is "not found", so path traversal such as `../../etc` cannot even reach the filesystem.
- File names are flat (no directories), limited to letters, digits, `_` and `-`, and must end in `.apn`, `.md` or `.txt`. A name is never joined from user-supplied path pieces.
- Writes are **atomic**: the content goes to a temporary file which is then renamed over the target, so a crash during a save cannot leave a half-written source file.
- Limits: at most 200 projects, 20 files per project, 200,000 bytes per file; `main.apn` cannot be deleted.

> **A REAL BUG AND ITS FIX**
>
> The first version validated names with regular expressions ending in `$`. In Python, `$` also matches before a trailing newline, so an application id such as `com.example.app\n` passed validation. All validation now uses `fullmatch`, which requires the whole string to match.

8.4 The database

backend/apppanrom/db.py (lines 18-41) - Schema

```
18  CREATE TABLE IF NOT EXISTS builds (
19    id TEXT PRIMARY KEY,
20    project_id TEXT NOT NULL,
21    status TEXT NOT NULL,
22    kind TEXT NOT NULL DEFAULT 'debug',
23    app_id TEXT,
24    app_name TEXT,
25    version TEXT,
26    created_at TEXT NOT NULL,
27    started_at TEXT,
28    finished_at TEXT,
29    duration_s REAL,
30    exit_code INTEGER,
31    failure_reason TEXT,
32    message TEXT,
33    artifact_name TEXT,
34    artifact_size INTEGER,
35    artifact_sha256 TEXT,
36    artifact_path TEXT,
37    artifact_deleted INTEGER NOT NULL DEFAULT 0,
38    log_path TEXT
39  );
40  CREATE INDEX IF NOT EXISTS builds_project ON builds(project_id, created_at);
41  """
```

The `Database` class wraps one SQLite connection behind a re-entrant lock (the build thread and request threads share it), enables write-ahead logging, and exposes three methods, `query`, `one` and `execute`, that all take parameters separately from SQL text. There is no string-built SQL anywhere, so SQL injection is not possible. A small migration adds the `signing` column if an older database lacks it.

8.5 Configuration

`config.py` reads settings only from environment variables, never from project source. A project's code therefore cannot change where keystores are kept, how long builds may run or which SDK is used.

Variable	Meaning
`APPANROM_DATA_DIR`	Where projects, the database, build logs, APKs and the Gradle cache live (default `./data`).
`ANDROID_HOME` / `ANDROID_SDK_ROOT`	Android SDK location.
`GRADLE_BIN`, `JAVA_HOME`	Gradle executable (8.7+) and JDK (17+).
`APPANROM_BUILD_TIMEOUT`	Seconds before a build is killed (default 900).
`APPANROM_KEYSTORE`, `_KEYSTORE_PASSWORD`, `_KEY_ALIAS`, `_KEY_PASSWORD`	Release signing. Passwords are excluded from the settings object's `repr()` so they cannot appear in logs.
`APPANROM_CORS_ORIGINS`	Allowed browser origins; a wildcard `*` is ignored on purpose.

9 The Studio (Frontend)

The Studio is a React single-page application written in TypeScript and built with Vite. It has two screens: the **Dashboard** (create, open, rename and delete projects) and the **Workspace**, which is where all work happens. The Workspace component (`Workspace.tsx`, 492 lines) owns the application state and coordinates the panels.

9.1 Layout and panels

Panel	Component	Function
Sidebar	`Palette`, `Outline`, `SettingsPanel`, `ReferencePanel`	Files, designer palette and outline, project settings and icon, language reference with examples.
Editor	`Editor`	CodeMirror 6 with the AppPanrom language and diagnostics.
Preview	`PhonePreview`, `DesignCanvas`	Run mode and design mode.
Bottom	`ConsolePanel`, `DiagnosticsPanel`, `Inspector`, `BuildPanel`	Console, problems, property inspector, builds.

9.2 The editor

CodeMirror 6 is configured with line numbers, history (undo and redo), a lint gutter, four-space indentation and a custom language. The language definition (`lang.ts`, 64 lines) is a small stream tokenizer: comments, strings with escapes, numbers, reserved words, `true/false/null` atoms and identifiers. An identifier that appears in the spec (a component, built-in function or configuration function) is coloured as a function, so the highlighter always matches the language because it reads the same spec the compiler uses.

frontend/src/components/Editor.tsx (lines 85-97) - replaceAll() makes the smallest possible change

```
85      replaceAll(text) {
86        const v = viewRef.current;
87        if (!v) return;
88        const old = v.state.doc.toString();
89        if (old === text) return;
90        let pre = 0;
91        while (pre < old.length && pre < text.length && old[pre] === text[pre]) pre++;
92        let suf = 0;
93        while (suf < old.length - pre && suf < text.length - pre && old[old.length - 1 - suf] === text[text.len
    gth - 1 - suf]) suf++;
94        v.dispatch({ changes: { from: pre, to: old.length - suf, insert: text.slice(pre, text.length - suf) }
    });
95      },
96      undo() { const v = viewRef.current; if (v) undo(v); },
97      redo() { const v = viewRef.current; if (v) redo(v); },
```

The `replaceAll` method is what makes designer edits feel live. Instead of replacing the whole document (which would reset scroll position and the undo history), it finds the common prefix and suffix of the old and new text and dispatches a change for only the differing middle. Designer edits therefore appear in the editor as a small insertion at the right place, and Ctrl+Z undoes them like typing.

9.3 Save, validate and the sync state

frontend/src/components/Workspace.tsx (lines 117-126) - validate() and onChange()

```
117   const validate = useCallback((src: string) => {
118     const seq = ++valSeq.current;
119     setSync('pending');
120     api.preview(projectId, src).then((r) => {
121       if (seq !== valSeq.current) return;
122       setDiags(r.diagnostics);
123       setChecked(true);
124       if (r.ok && r.ir) { setDesignIr(r.ir); setSync('ok'); } else setSync('errors');
125     }, () => undefined);
126   }, [projectId]);
```

- **Autosave** runs 1,200 ms after the last keystroke; the toolbar shows "Unsaved changes", "Saving" or "All changes saved". Leaving the page with unsaved changes asks for confirmation.
- **Validation** runs 450 ms after the last keystroke. A sequence number guards against out-of-order responses: if a slower, older response arrives after a newer one, it is ignored.
- The **sync state** (`ok` , `pending` , `errors`) tells the designer whether the code currently compiles. Designer operations are disabled unless it is `ok` , because the designer must work from a valid program.

9.4 The sandboxed preview

The preview must run user programs in the browser, which is the most security-sensitive part of the Studio. The IR is rendered inside an `iframe` whose `sandbox` attribute allows scripts but **not** same-origin access, so the app inside cannot read the Studio's cookies, storage or DOM. A Content-Security-Policy meta tag inside the frame blocks everything except inline scripts and styles and `https:` / `data:` images.

frontend/src/previewDoc.ts (lines 20-31) - Building the srcdoc of the preview

```
<script>
(function () {
  var TOKEN = ${jsonForScript(token)};
  var data = ${jsonForScript(store)};
  function post(m) { m.token = TOKEN; parent.postMessage(m, '*'); }
  var storage = {
    getItem: function (k) { return Object.prototype.hasOwnProperty.call(data, k) ? data[k] : null; },
    setItem: function (k, v) { data[k] = String(v); post({ type: 'ap-store', op: 'set', key: k, value: String(v) }); },
    removeItem: function (k) { delete data[k]; post({ type: 'ap-store', op: 'remove', key: k }); },
    keys: function () { return Object.keys(data); }
  };
  var ir = ${jsonForScript(ir)};
```

The frame has no storage of its own because of the sandbox, so storage is **bridged**: the runtime in the frame calls the storage object, which updates a local copy and sends a `postMessage` to the Studio, which persists it in its own `localStorage` keyed by project. When the preview starts, the Studio passes the stored data in. Messages carry a random **token** created per run, and the Studio ignores any message whose source window or token does not match, so a stale or foreign frame cannot write data.

frontend/src/components/PhonePreview.tsx (lines 40-54) - Receiving messages from the frame

```
  useEffect(() => {
    const on = (e: MessageEvent) => {
      const m = e.data;
      if (e.source !== frame.current?.contentWindow || !m || m.token !== token) return;
      if (m.type === 'ap-log' && typeof m.text === 'string') onLog(m.level === 'info' || m.level === 'warning' || m.level === 'error' ? m.level : 'debug', m.text.slice(0, 2000));
      if (m.type === 'ap-store' && typeof m.key === 'string') {
        const data = loadStore(projectId);
        if (m.op === 'set') data[m.key] = String(m.value);
        else delete data[m.key];
        saveStore(projectId, data);
      }
    };
    window.addEventListener('message', on);
    return () => window.removeEventListener('message', on);
  }, [token, projectId, onLog]);
```

The IR and the runtime are embedded as inline scripts. The helper `jsonForScript` escapes `<`, U+2028 and U+2029 so that no string inside the IR can close the script tag early.

9.5 The design canvas

frontend/src/components/DesignCanvas.tsx (lines 30-45) - Mounting the runtime in a shadow root

```
  useEffect(() => {
    const el = host.current!;
    const shadow = el.shadowRoot ?? el.attachShadow({ mode: 'open' });
    const AP = getRuntime();
    const c = AP.mount(first.current.ir, shadow, {
      namespace: 'design',
      storage: AP.memoryStorage(),
      log: () => undefined,
      design: {
        selectedLine: first.current.selectedLine,
        onSelect: (l: number | null) => cb.current.onSelect(l),
        onDrop: (i: DropInfo) => cb.current.onDrop(i),
        onTree: (t: DesignTree) => cb.current.onTree(t),
      },
    });
    ctrl.current = c;
```

The canvas mounts the runtime in design mode in a shadow root and does nothing else. It does not modify state; it converts runtime callbacks (select, drop, outline tree) into calls to the Workspace, which turns them into designer operations. The next IR is passed to the existing runtime through `update(ir)`, which creates a new runtime instance but keeps the same DOM root, the current screen and the scroll position, so a drag does not flash the whole phone.

frontend/src/components/Workspace.tsx (lines 186-200) - applyDesignEdit(): the heart of the designer in the UI

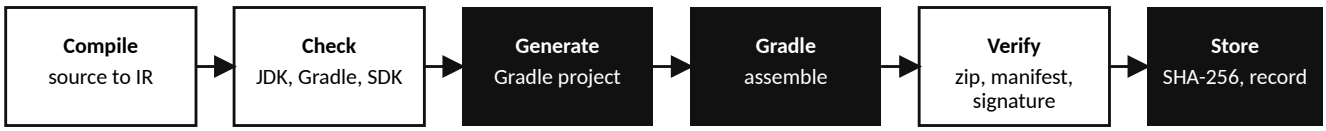
```
186   const applyDesignEdit = useCallback(async (op: Record<string, unknown>, quiet = false): Promise<string | nu
    ll> => {
187     if (syncRef.current !== 'ok') {
188       const m = 'Fix the errors in your code first; the designer needs code that compiles.';
189       if (!quiet) setDesignError(m);
190       return m;
191     }
192     try {
193       const r = await api.designEdit(projectId, text.current, op);
194       designEditing.current = true;
195       editor.current?.replaceAll(r.source);
196       designEditing.current = false;
197       window.clearTimeout(valTimer.current);
198       validate(r.source);
199       setSelected(r.line);
200       if (r.line) editor.current?.reveal(r.line);
```

9.6 The build panel

The build panel loads `/api/system/status` to show which prerequisites are missing, offers the three build kinds (release kinds are disabled until a signing key is configured), starts a build and then streams the log. Streaming is a simple poll: every second it requests the log from a byte offset, appends the new text, and refreshes the build record. When the build ends it fetches once more so that the final lines are never lost. A **Download** button appears only when the build record contains an artifact.

10 The Android Pipeline

This chapter explains how a program becomes an installable app. The principle is that the Android part is as thin as possible: the generated project is a small native shell that hosts the runtime in a WebView and holds the app's IR as data.



Any failing step ends the build as "failed" with a reason; the result is never reported as success without an artifact.

Figure 10.1: The build pipeline.

10.1 The template

The template (`android-template/`) is a complete Gradle project with placeholders. `app/build.gradle` uses Android Gradle Plugin 8.5.2, compile and target SDK 34, minimum SDK 24 (Android 7.0), Java 17 and one dependency, `androidx.webkit` .

android-template/app/build.gradle (lines 1-17) - Module build file with placeholders

```
plugins {
    id 'com.android.application'
}

android {
    // Fixed code namespace; the user-facing application id is set below.
    namespace 'com.apppanrom.shell'
    compileSdk 34

    defaultConfig {
        applicationId "__APPLICATION_ID__"
        minSdk 24
        targetSdk 34
        versionCode __VERSION_CODE__
        versionName "__VERSION_NAME__"
    }

```

The **namespace** (the Java package of the shell code) is fixed as `com.apppanrom.shell`, while the user's **application id** (the identity of the app on a phone) is substituted from the program. Keeping them separate means the Java code never needs to be renamed.

The only native code

android-template/app/src/main/java/com/apppanrom/shell/MainActivity.java (lines 24-46) - MainActivity

```
        final WebViewAssetLoader loader = new WebViewAssetLoader.Builder()
                .addPathHandler("/assets/", new WebViewAssetLoader.AssetsPathHandler(this))
                .build();
        web = new WebView(this);
        setContentView(web);
        WebSettings s = web.getSettings();
        s.setJavaScriptEnabled(true);
        s.setDomStorageEnabled(true);
        s.setAllowFileAccess(false);
        s.setAllowContentAccess(false);
        s.setMixedContentMode(WebSettings.MIXED_CONTENT_NEVER_ALLOW);
        web.setWebViewClient(new WebViewClient() {
            @Override
            public WebResourceResponse shouldInterceptRequest(WebView view, WebResourceRequest request) {
                return loader.shouldInterceptRequest(request.getUrl());
            }

            @Override
            public boolean shouldOverrideUrlLoading(WebView view, WebResourceRequest request) {
                return !request.getUrl().toString().startsWith(ORIGIN + "/assets/");
            }
        });
        web.loadUrl(ORIGIN + "/assets/www/index.html");
```

The entire native app is one Activity. Key security decisions:

- Files are served from the APK's `assets` folder through `WebViewAssetLoader` over a virtual HTTPS origin (`appassets.androidplatform.net`). Access to `file://` and `content://` URLs is disabled, and mixed content is never allowed.
- Navigation to any URL outside `/assets/` is blocked. The app cannot be redirected to an external web page.
- There is **no JavaScript-to-native bridge** (`addJavascriptInterface` is never called). Even a flaw in the runtime could not call into Android APIs.
- The system Back button asks the runtime whether it has a screen to go back to (`window.__apBack()`) and closes the app only when it does not.

The manifest

android-template/app/src/main/AndroidManifest.xml (lines 1-19) - Manifest template

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android">
__PERMISSIONS__
    <application
        android:allowBackup="false"
__ICON_ATTR__        android:label="@string/app_name"
        android:theme="@style/AppTheme"
        android:usesCleartextTraffic="false">
        <activity
            android:name=".MainActivity"
            android:exported="true"
            android:configChanges="orientation|screenSize|keyboardHidden|smallestScreenSize|screenLayout|uiMode">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />
                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
    </application>
</manifest>
```

`allowBackup="false"` keeps app data out of device backups; `usesCleartextTraffic="false"` forbids plain HTTP. The two placeholders are filled by the generator: `__PERMISSIONS__` becomes one `uses-permission` line per permission found in the IR, and `__ICON_ATTR__` becomes the icon attribute if the project has an icon.

10.2 The generator

backend/apppanrom/android/generate.py (lines 74-91) - generate_project

```
def generate_project(ir: dict, dest: Path, icon: Path | None = None) -> None:
    """Writes a complete Gradle project into `dest` (must not exist). Values come from the validated IR only."""
    app = ir["app"]
    if not APP_ID_RE.fullmatch(app["id"]) or not VERSION_RE.fullmatch(app["version"]):
        raise GenerateError("Application id or version failed validation.")
    if dest.exists():
        raise GenerateError("Build directory already exists.")
    shutil.copytree(TEMPLATE_DIR, dest)
    subs = {
        "__APPLICATION_ID__": app["id"],
        "__VERSION_CODE__": str(version_code(app["version"])),
        "__VERSION_NAME__": app["version"],
    }
    gradle = dest / "app" / "build.gradle"
    text = gradle.read_text(encoding="utf-8")
    for k, v in subs.items():
        text = text.replace(k, v)
    gradle.write_text(text, encoding="utf-8")
```

The generator copies the template into a new, empty directory and then writes the per-app values. Everything it writes comes from the **validated IR**, and it re-validates the two values that matter most. Steps:

1. Re-check the application id and version against their patterns; refuse if the target directory already exists.
2. Substitute the application id, version name and a version code computed as `major * 1,000,000 + minor * 1,000 + patch` (so version 1.2.3 becomes code 1,002,003, and a higher version always gives a higher code, as Android requires for updates).
3. Write the permissions from the IR into the manifest after re-checking each against `android.permission.[A-Z_]{3,40}`.
4. If an icon exists, validate it again and copy it as the launcher icon.
5. Write the app name into `strings.xml` through `xml_text()`, which removes control characters and escapes `& < > ' " \` and a leading `@` or `?`.
6. Write `index.html`, copy the runtime into `assets/www/`, and write `app.js` containing the IR. In the JSON, `</` is replaced by `<\/` so no string can close a script tag.

The page includes a Content-Security-Policy of `default-src 'none'; script-src 'self'; style-src 'self' 'unsafe-inline'; img-src 'self' https: data:`. Scripts may only come from the app's own files (no inline scripts, no remote scripts); images may come from the app, from HTTPS or from data URIs; everything else, including network requests, is blocked.

10.3 Prerequisite detection

`prereqs.py` answers "can this machine build an APK?" before any work starts. It runs three checks and returns, for each, whether it passed, a detail line and a "fix" instruction:

- **Java:** finds `java` (from `JAVA_HOME` or `PATH`), parses `java -version` and requires version 17 or later.
- **Gradle:** finds the executable (`GRADLE_BIN` or `PATH`), parses `gradle --version` and requires 8.7 or later, the minimum for the Android plugin in use.
- **Android SDK:** requires `ANDROID_HOME` to exist, to contain `platforms/android-34`, and to have build-tools of version 34 or higher.

The Studio shows these results in the Build tab, and the build manager logs them as the first step of a build, so a missing tool is reported as a clear instruction rather than a Gradle error many minutes in. Subprocesses for these checks run with a reduced environment (only `PATH`, `HOME`, `JAVA_HOME`, `JAVA_TOOL_OPTIONS`) and no standard input.

10.4 The build manager

backend/apppanrom/android/builder.py (lines 203-235) - The build run

```
203     def _run(self, build_id: str, project_id: str, ir: dict, kind: str, icon: Path | None) -> None:
204         started = time.monotonic()
205         work = self.s.builds_dir / build_id / "work"
206         try:
207             self.db.execute("UPDATE builds SET status='running', started_at=? WHERE id=?", (_now(), build_i
    d))
208             pre = check_prerequisites(self.s, use_cache=False)
209             if not pre["ok"]:
210                 text = format_problems(pre)
211                 self._log(build_id, text)
212                 self._fail(build_id, started, "prerequisites_missing", "Android build prerequisites are missi
    ng; see the log.")
213                 return
214             task, rel_out, suffix, is_release = KINDS[kind]
215             if is_release:
216                 sign = check_signing(self.s)
217                 if not sign["configured"]:
218                     self._log(build_id, f"{sign['detail']}\n  fix: {sign['fix']}")
219                     self._fail(build_id, started, "signing_not_configured", "Release signing is not configure
    d; see the log. No unsigned release is ever produced.")
220                     return
221             self._log(build_id, "[apppanrom] prerequisites OK; generating Android project")
222             try:
223                 generate_project(ir, work, icon)
224             except GenerateError as e:
225                 self._fail(build_id, started, "invalid_project", str(e))
226                 return
227             code, timed_out = self._gradle(build_id, work, task, is_release)
228             if timed_out:
229                 self._fail(build_id, started, "timeout", f"Build exceeded the {self.s.build_timeout_s}s time
    limit and was stopped.", code)
230                 return
231             if code != 0:
232                 self._fail(build_id, started, "gradle_failed", f"Gradle exited with code {code}; see the lo
    g.", code)
233                 return
234             out = work / rel_out
235             problem = verify_aab(out) if kind == "release_aab" else verify_apk(out)
```

A build is a row in the `builds` table, a log file and, if it succeeds, an artifact. The lifecycle is **queued** then **running** then **succeeded** or **failed**, and a failure always records a reason: `prerequisites_missing`, `signing_not_configured`, `invalid_project`, `gradle_failed`, `timeout`, `artifact_invalid`, `interrupted` or `internal_error`.

- **One build at a time.** A thread pool with one worker serialises builds, because Gradle is heavy and builds share a cache.
- **Isolation.** Each build has its own directory under `builds/<id>/work`. The directory is deleted at the end unless `APPANROM_KEEP_BUILD_DIRS=1` is set for debugging.
- **Restart safety.** At startup, rows left in `queued` or `running` (the server died mid-build) are marked failed with the reason `interrupted`, so the UI never shows a build that is "running" forever.
- **Never stuck.** The whole run is wrapped so that any unexpected exception records `internal_error` instead of leaving the row in `running`.

Running Gradle safely

backend/apppanrom/android/builder.py (lines 290-309) - Starting and supervising Gradle

```
290      def _gradle(self, build_id: str, work: Path, task: str, release: bool) -> tuple[int, bool]:
291          exe = gradle_bin(self.s)
292          cmd = [exe, "--no-daemon", "--console=plain", "-p", str(work), task]
293          env = {k: os.environ[k] for k in PASSTHROUGH_ENV if k in os.environ}
294          env["GRADLE_USER_HOME"] = str(self.s.gradle_home)
295          env["ANDROID_HOME"] = str(self.s.android_home)
296          env["ANDROID_SDK_ROOT"] = str(self.s.android_home)
297          if self.s.java_home:
298              env["JAVA_HOME"] = str(self.s.java_home)
299          if release:  # secrets go to the Gradle process environment only: never into files, argv or logs
300              env["APPANROM_SIGN_STORE_FILE"] = str(self.s.keystore_path)
301              env["APPANROM_SIGN_STORE_PASSWORD"] = self.s.keystore_password or ""
302              env["APPANROM_SIGN_KEY_ALIAS"] = self.s.key_alias or ""
303              env["APPANROM_SIGN_KEY_PASSWORD"] = self.s.effective_key_password
304          self._log(build_id, f"[apppanrom] running: gradle --no-daemon --console=plain {task}")
305          log_path = self.log_path(build_id)
306          proc = subprocess.Popen(cmd, cwd=work, env=env, stdin=subprocess.DEVNULL, stdout=subprocess.PIPE,
307                                  stderr=subprocess.STDOUT, start_new_session=True)
308          with self._lock:
309              self._procs[build_id] = proc
```

- Gradle is started with an **argument array** (not a shell string), so no part of any project value can be interpreted as a command.
- It receives only an **allow-list of environment variables** (`PATH`, `HOME`, `LANG`, proxy and certificate settings, Java and Android locations). Other secrets in the developer's environment never reach the build.
- The process is started in its own session, so on timeout the whole process group (Gradle, its daemons, child processes) can be killed with one signal.
- A **timer** kills the build after the timeout (default 900 s). A **log cap** of 2,000,000 bytes stops a runaway build from filling the disk; the log notes when output was discarded.
- Output is read line by line and flushed to the log file immediately, so the Studio can stream it live.

10.5 Verifying the result

backend/apppanrom/android/builder.py (lines 43-60) - verify_apk

```
43  def verify_apk(path: Path) -> str | None:
44      """Returns a problem description, or None if the file is a plausible APK."""
45      try:
46          size = path.stat().st_size
47      except OSError:
48          return "APK file was not produced."
49      if size < MIN_APK_BYTES:
50          return f"APK is only {size} bytes, which is too small to be a real app."
51      try:
52          with zipfile.ZipFile(path) as z:
53              names = set(z.namelist())
54      except zipfile.BadZipFile:
55          return "Output is not a valid APK (zip) file."
56      if "AndroidManifest.xml" not in names:
57          return "APK has no AndroidManifest.xml."
58      if not any(n.endswith("apppanrom-runtime.js") for n in names):
59          return "APK does not contain the AppPanrom runtime."
60      return None
```

Gradle exiting with code 0 is not treated as proof. The produced file must exist, be at least 50,000 bytes, be a valid zip, contain `AndroidManifest.xml`, and contain the AppPanrom runtime. A bundle (AAB) is checked for `BundleConfig.pb` and its base manifest. Only then is the file copied to the artifact store, hashed with SHA-256, and recorded. The download endpoint re-checks that the stored file still exists and still has the recorded size, and that its resolved path lies inside the artifacts directory. Old artifacts are pruned to the latest 10 per project.

> **WHY VERIFICATION MATTERS**
>
> It is tempting for a tool to print "Build successful" whenever the last command did not fail. A tool for learners must be honest: if the file is missing, empty or not a real app, the learner must be told so. Throughout the pipeline, "succeeded" has exactly one meaning: a verified artifact exists.

10.6 Release signing

A debug APK is signed with the automatically generated Android debug key and can be installed for testing but not published. A **release** build must be signed with the developer's own key. The design treats the key material as the most sensitive data in the project:

android-template/app/build.gradle (lines 18-37) - Signing configuration read from the environment

```
18      // Release signing material is read from the environment of the Gradle process only (never from files in
    this project).
19      signingConfigs {
20          release {
21              def storeFilePath = System.getenv("APPANROM_SIGN_STORE_FILE")
22              if (storeFilePath) {
23                  storeFile file(storeFilePath)
24                  storePassword System.getenv("APPANROM_SIGN_STORE_PASSWORD")
25                  keyAlias System.getenv("APPANROM_SIGN_KEY_ALIAS")
26                  keyPassword System.getenv("APPANROM_SIGN_KEY_PASSWORD")
27              }
28          }
29      }
30
31      buildTypes {
32          release {
33              minifyEnabled false
34              // Without the environment above this fails loudly instead of producing an unsigned "release".
35              signingConfig signingConfigs.release
36          }
37      }
```

- The keystore path, passwords and alias exist only as environment variables of the backend process. They are not stored in the database, not written to any file in the generated project, not placed on a command line, and not

logged. The settings object hides passwords from its text representation.

- The build manager passes them only to the Gradle process environment. The template reads them with `System.getenv`. If they are missing, the release build fails loudly: **an unsigned "release" is never produced**.
- Release kinds fail early with `signing_not_configured` and a fix instruction if the key is not set up, before any Gradle time is spent.
- After the build, the signature is **verified independently** with `apksigner verify --print-certs` (APK) or `jarsigner -verify` (AAB). The result is rejected if the signing certificate is the Android debug key. If the verifier is not installed, the record says "not independently verified" instead of claiming more than is known.
- A command-line helper creates a key safely: `python -m apppanrom.cli make-keystore path --alias name` asks for the password with `getpass`, passes it to `keytool` through an environment variable (not an argument), and sets file permissions to owner-only.

10.7 App icon

The icon validator (`icon.py`, 35 lines) parses the PNG by hand, with no image library: it checks the 8-byte signature, the `IHDR` chunk header and its CRC-32, requires a square image between 192 and 1,024 pixels, a size of at most 512,000 bytes, and a proper `IEND` trailer. A snapshot of the icon is copied into the build directory at submission, so editing the icon while a build is queued cannot change that build.

10.8 Continuous integration: real builds

The development machine used for most of the work could not reach Google's download servers, so a real Gradle build could not run there. The project therefore includes a GitHub Actions workflow that builds the example apps on GitHub's runners, which have internet access and the Android SDK.

`.github/workflows/android-apk.yml (lines 58-76) - Generate, build and verify`

```
58        - name: Compile the AppPanrom program and generate the Android project
59          working-directory: backend
60          run: python -m apppanrom.cli android-project "../$APP_FILE" "$RUNNER_TEMP/app-${{ matrix.name }}"
61
62        - name: Build the debug APK with Gradle
63          working-directory: ${{ runner.temp }}/app-${{ matrix.name }}
64          run: gradle --no-daemon --console=plain assembleDebug
65
66        - name: Verify the APK is real
67          working-directory: backend
68          run: |
69            python - <<'PY'
70            import os, pathlib, sys
71            from apppanrom.android.builder import verify_apk
72            apk = pathlib.Path(os.environ["RUNNER_TEMP"]) / "app-${{ matrix.name }}/app/build/outputs/apk/debu
    g/app-debug.apk"
73            problem = verify_apk(apk)
74            print(apk, apk.stat().st_size if apk.exists() else "missing", "->", problem or "OK")
75            sys.exit(1 if problem else 0)
76            PY
```

The workflow uses a build matrix with one job per example app. Each job installs JDK 17 and Gradle 8.9, makes sure the SDK platform and build-tools for API 34 are installed, runs the CLI command `android-project` (compile, then generate), runs `gradle assembleDebug`, and then runs the same `verify_apk` function the backend uses. A final job collects the APKs, writes a README with their SHA-256 checksums, and publishes them to a dedicated `apk-builds` branch so they can be fetched with plain `git`. Seven apps are built this way. Each APK is about 0.47 MB.

HONEST STATUS

The APKs are built and structurally verified on real infrastructure. They are debug-signed and have **not yet been installed and exercised on physical devices**. Device testing is the first item of future work.

11 Security Model

AppPanrom executes code written by its users, so it is designed as if that code were hostile. It is also a local single-user tool, so some protections that a public service would need are intentionally out of scope. This chapter states both clearly.

11.1 What is protected, and how

Threat	Mitigation
User code runs arbitrary Python or JavaScript	User code is never converted to Python or JavaScript. It is parsed with a fixed grammar and interpreted by walking a validated IR. Function calls are resolved against a user-function table and a fixed built-in table; there is no way to reach host objects. A test feeds raw Python, shell and JavaScript to the compiler and the runtime and verifies that nothing executes.
Endless loops and memory exhaustion	Interpreter step limit (1,000,000 per entry point), call depth limit (100), list and text size limits. These errors cannot be caught by `try`.
Script injection through app text	The IR is embedded in a script as JSON with `<` escaped, so no value can close the tag. All app text is placed into the DOM as text, never as HTML. Style values are validated and applied as individual CSS properties.
The preview reading the Studio's data	Sandboxed iframe without same-origin access, restrictive CSP, storage bridged through token-checked `postMessage`.
Path traversal and file abuse	Generated project ids, flat validated file names, `fullmatch` patterns, size and count limits, atomic writes. The download endpoint checks the resolved path stays inside the artifacts directory.
SQL injection	All queries are parameterised. A test sends injection strings as ids and verifies nothing happens.
Command injection in builds	Gradle is started with an argument array. Project values reach the build only as validated data (application id, version) and as data files (IR). The environment passed to Gradle is an allow-list.
Leaking signing keys	Keys come only from environment variables, go only to the Gradle process, are hidden from `repr()`, never stored, logged or committed (the repository ignores keystores). A test checks that debug builds never receive signing secrets.
A hostile app on the phone	No JavaScript bridge, no `file://` access, no cleartext traffic, navigation restricted to local assets, CSP forbids remote scripts, permissions declared only for features actually used.
Cross-origin web requests	CORS allows only listed development origins; a wildcard is ignored.
Information leaks in errors	A catch-all handler returns a generic message; no stack traces or paths.

11.2 Known limits of the model

- There is **no authentication**. The server is meant to be run on the developer's own machine and must not be exposed to the internet as it is.
- A public deployment would additionally need user accounts and per-user isolation, HTTPS, quotas and rate limits, and running Gradle builds inside a locked-down container (Gradle runs plugin code from the template's dependencies).
- App-local storage is not encrypted; `random_number` is not cryptographically secure. Both are documented.
- The language has no network access yet. When it is added, requests must go through a server-side proxy that validates destinations, to avoid server-side request forgery.

12 Testing and Quality

A project that claims to be honest needs evidence. The repository has four layers of automated tests; one command (`make test`) runs the first three, and `make e2e` runs the fourth.

Suite	Tests	What it proves
`backend/tests/test_language.py`	28 + 1	Syntax errors have positions; suggestions; undefined variables; scoping; screen and component rules; handler validation; style validation and scoping; render cannot mutate globals; literal type mismatches; size and nesting limits; **every shipped example compiles with zero diagnostics**; the language document mentions every spec item.
`test_api.py`	12	CRUD, uniform errors, file limits, path traversal, persistence across restart, CORS restriction, SQL-injection ids, spec endpoint, design endpoints.
`test_designer.py`	26	Insert/move/delete/duplicate at every position; property, style and event edits; invalid values rejected; unparser round trips; 150 randomised edits keep the file valid.
`test_builds.py` , `test_signing_icon.py`	31	Build manager with a Gradle test double: status transitions, log capture and cap, timeout kill, rejection of tiny or corrupt APKs, artifact pruning, restart recovery, project deletion; generated project contents; identifier safety; icon validation; signing configuration; real `keytool` and `jarsigner` for the release bundle path.
`runtime/test/runtime.test.mjs`	36	The runtime implements exactly the functions and components in the spec; expressions and typing; control flow; try/catch; limits; pure rendering; inputs; styles; storage and persistence; navigation; device features; code injection attempts; and full behavioural tests of five real apps.
`frontend/src/previewDoc.test.ts`	1	The script-embedding helper cannot be tricked into closing the script tag.
`frontend/e2e/run.mjs`	15 steps	A real backend serving the built Studio and a real Chromium: empty dashboard, create project, live diagnostics, run and interact, syntax error state, a styled two-screen app with navigation and storage, honest build failure without an SDK, persistence after reload, settings validation, rename and delete, a complete drag-and-drop design session, icon upload, release options disabled without a key, and the generated Android web bundle running under the same CSP as the APK.

Together the unit and integration suites contain **156 tests** (120 backend, 36 runtime), all passing.

12.1 The Gradle test double

Because the build manager must be testable without the Android SDK, the tests supply a small stand-in "Gradle" script that behaves like the real tool for the purposes of the pipeline: it writes a plausible APK or fails, sleeps to trigger timeouts, or floods the output to test the log cap. This proves the *mechanics* (status transitions, log capture, timeouts, verification, download gating). It deliberately does not claim to prove that the Android template compiles; that is what the continuous-integration build on a real runner demonstrates.

12.2 Testing real apps, not just features

The runtime tests include one behavioural test per example app. They drive the app through its real handlers exactly as a user would, then check the displayed text:

- **Calculator:** 18 key sequences covering digits, operators, decimals, negation, backspace, chained operations and division by zero.
- **Daily Planner:** seeded dashboard, sorting by priority, filters, search, completing and deleting, form validation, duplicates, statistics, accent colour change, persistence across a restart, reset and wipe.
- **Pocket Ledger:** totals, validation of the amount, budget status, undo, persistence.
- **Brain Quiz:** answer locking, scoring, feedback, best score, persistence.
- **Split & Tip and Hydrate:** arithmetic, unit conversion, adding and undoing water, goals, and day rollover with a controllable clock.

> CONTROLLABLE TIME
>
> Testing "a new day starts from zero" must not wait a day. The runtime accepts an optional `now()` function in its host object, used by `now_iso()`. Tests pass a fake clock; the real app uses the system clock.

12.3 What the tests caught

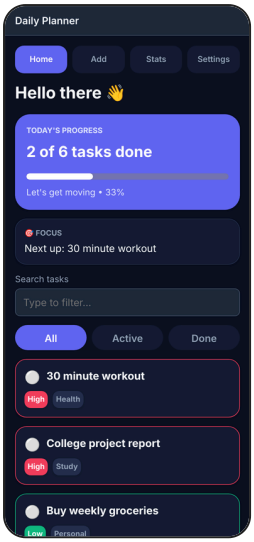
The value of tests shows in the defects they exposed during development. Examples: the trailing-newline validation bug (Section 8.3); designer edit ordering at equal positions (Section 7.3); a release key password that defaulted only when read from the environment, fixed by a single `effective_key_password` property; release build options that could be selected before the signing status had loaded in the UI; and the runtime global being hidden by the bundler's module wrapper, fixed by loading the runtime as a plain script.

13 Example Apps

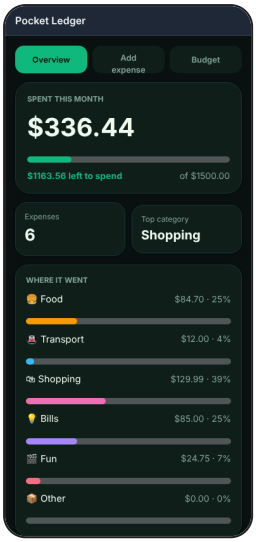
The `examples/` folder contains nine programs that double as documentation, test material and demonstrations. Each one compiles with zero diagnostics, which a test enforces. Seven are built as APKs by continuous integration.

App	Lines	What it demonstrates
`vanakkam.apn`	40	The Tanglish "hello": an input, a button, a message dialog and a click counter.
`counter.apn`	27	Global state, conditional messages, progress bar.
`notes.apn`	30	A list kept in a variable, a loop that renders it.
`calculator.apn`	177	A four-function calculator with decimals, sign change, backspace and error handling for division by zero. One handler per key.
`daily_planner.apn`	353	The most complex app: dashboard with progress, search, filters, priority sorting, add form, statistics per priority, accent-colour settings, local persistence.
`pocket_ledger.apn`	272	Expense tracker with budget progress, per-category bars coloured through dynamic styles, recent list, undo, persistence.
`brain_quiz.apn`	198	Ten-question quiz with locked answers, colour feedback, fun facts, best score and games played.
`split_and_tip.apn`	176	Bill splitter with tip presets and a unit converter on a light theme; live recalculation from dropdowns and number inputs.

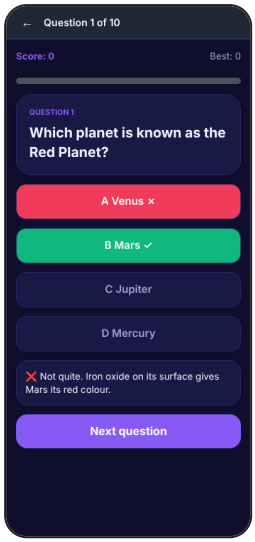
`hydrate.apn`	160	Water tracker with quick-add buttons, goal selection, undo, and a seven-day history with automatic day rollover.



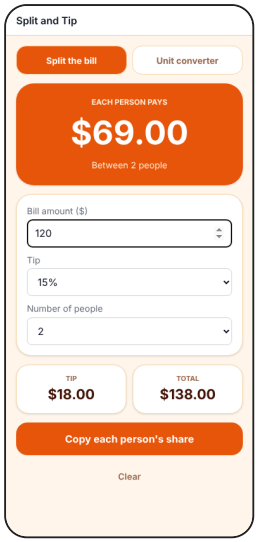
Daily Planner



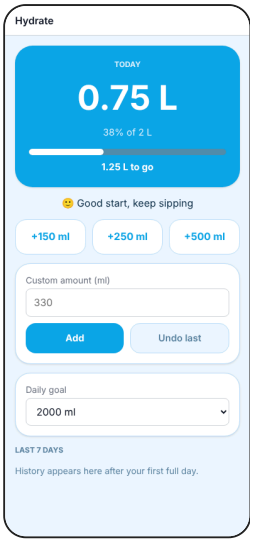
Pocket Ledger



Brain Quiz



Split & Tip



Hydrate

13.1 Patterns the examples use

State plus pure helper functions

Because a screen may not change state while drawing, apps compute derived values in helper functions that only read variables. The Pocket Ledger, for example, formats money, totals categories and chooses a colour from the budget status without any side effects:

examples/pocket_ledger.apn (lines 78-90) - Formatting money without side effects

```
78
79  # ------------------------------------------------------------------ numbers
80  function money(n):
81      mari parts = text_split(str(round(n, 2)), ".")
82      enna_na length(parts) == 1:
83          return "$" + parts[0] + ".00"
84      enna_na text_length(parts[1]) == 1:
85          return "$" + parts[0] + "." + parts[1] + "0"
86      return "$" + parts[0] + "." + parts[1]
87
88  function total_spent():
89      mari t = 0
90      ovvoru e in expenses:
```

Handlers that validate and persist

examples/pocket_ledger.apn (lines 155-168) - A handler: validate, change state, save, navigate

```
155 function save_expense():
156     mari amount = input_value("amount")
157     enna_na amount == null or amount <= 0:
158         toast_kaattu("Enter an amount greater than zero")
159         return
160     mari note = text_trim(input_value("note"))
161     enna_na note == "":
162         note = input_value("category")
163     append(expenses, {"amount": amount, "cat": input_value("category"), "note": note})
164     persist()
165     input_set("amount", null)
166     input_set("note", "")
167     toast_kaattu("Expense saved")
168     screen_ku_poo("home")
```

The handler reads the number input (which is `null` when empty), validates it, appends to the list, saves the list, clears the inputs and navigates home. The redraw that follows shows the new total.

Dynamic styles

Because style values may be expressions, the interface can reflect state without extra components. The progress bar's colour is a function of how much of the budget is used, and each category bar takes the category's own colour. The compiler checks only literal values, so the runtime validates the colours on every draw and ignores a bad one with a warning.

Day rollover with a stored date

The Hydrate app records the date of the last use. Each action first calls `roll_day()`: if the stored date differs from today, yesterday's total is appended to the history (kept to seven entries), today's counters are reset and the new date is saved. This pattern shows how to build time-dependent behaviour with only `now_iso()` and local storage.

13.2 Design notes for the example apps

Each example is written entirely in English text (apart from keywords) because the apps are meant to be used by anyone, not only by Tamil speakers; Tanglish is the language of *programming*, not necessarily of the app's interface. Colours are held in small functions (`bg()` , `card_bg()` , `accent()`) so that a theme can be changed in one place, and the apps use emoji as lightweight icons, since the component set deliberately has no icon component.

14 Limitations and Future Work

14.1 Honest status

Area	Status
Language, compiler, runtime, designer, Studio	Implemented and tested (Chapters 4-9, 12).
Debug APK builds	Built on real infrastructure by CI for seven apps; structurally verified. **Not yet installed on physical devices.**
Release APK and AAB	Implemented: key from environment, signature verified with `apksigner` / `jarsigner` . The AAB path is exercised in tests with a real `keytool` and `jarsigner` ; a full release build with a real SDK has not been demonstrated yet.
Network APIs, camera, location, notifications, dialogs	Not implemented. The compiler reports them as "planned but not implemented".
Authentication, multi-user	Not implemented; the tool is single-user and local.

14.2 Known limitations

- The whole screen is rebuilt after every event. This is fast enough for small apps but would not scale to very large screens.
- The designer has no free positioning or resize handles. Layout is flex-based: components are reordered and nested, and sizes are set in the inspector. Selection is cleared when the user types in the code (line numbers shift).
- Only `main.apn` is compiled; other files in a project are notes.
- Builds run one at a time; the build list shows the latest 50 builds.

- No `break` / `continue`, no modules, no first-class functions; every handler takes no arguments, which makes repeated buttons (such as calculator keys) verbose.

14.3 Future work

1. **Device testing** on a range of Android versions and screen sizes; add an emulator-based test to CI.
2. **Release flow** end to end with a real keystore and Play Store bundle upload checks.
3. **Network access** through validated server-side proxying (`api_get` and friends), then runtime-permission features (camera, location, notifications) with explicit permission prompts.
4. **Handlers with arguments** (for example, "which list item was tapped"), which would make list-heavy apps much shorter.
5. **More themes and styling** (gradients, shadows, custom fonts) and a theme editor.
6. **More regional keyword sets**, since the keyword table is data and the architecture does not depend on Tanglish specifically.
7. **Classroom features**: shareable project links, exercises with automatic checking, and an offline-installable Studio.

A Appendix: Reference Tables

A.1 Built-in functions

Signature	Description
`message_kaattu(value)`	Modal message with an OK button.
`toast_kaattu(value)`	Short transient message.
`debug_log, warning_kaattu, error_kaattu (value)`	Write a console line at that level.
`int(value)` , `float(value)`	Convert a number or numeric text to an integer or a number; error otherwise.
`str(value)` , `bool(value)` , `type_of(value)`	Text form; truthiness; "number", "text", "bool", "list", "dict" or "null".
`text_length, text_upper, text_lower, text_trim`	Text length, case conversion, trimming whitespace.
`text_replace(text, old, new)`	Replace all occurrences; `old` may not be empty.
`text_split(text, sep)` , `text_join(items, sep)`	Split into a list; join a list into text.
`text_contains(text, part)`	True if `part` occurs in `text` .
`length(list_or_dict)`	Number of items (for text use `text_length`).
`append(list, value)` , `remove(list, index)`	Change a list in place; `remove` returns the removed value.
`sort(list)`	New sorted list of all numbers or all text.
`range(stop)` , `range(start, stop)`	List of whole numbers.
`dict_get(d, key, default)` , `dict_keys(d)`	Safe lookup; list of keys.

`min, max, sum, round(value, digits), abs`	Numeric helpers; `round` accepts 0-10 digits.
`json_encode(value)`, `json_decode(text)`	Convert to and from JSON text.
`now_iso()`	Current UTC time as ISO 8601 text.
`random_number(low, high)`, `random_choice(list)`	Random integer (both ends included); random item. Not cryptographically secure.
`input_value(name)`, `input_set(name, value)`	Read or write an input component's value.
`data_save_pannu(key, value)`, `data_edu(key, default)`	Save and read JSON-compatible data.
`data_delete_pannu(key)`, `data_clear_pannu()`	Remove one key or all of the app's data.
`screen_ku_poo(name)`, `screen_back_poo()`	Navigate to a screen; go back.
`screen_data_set(key, value)`, `screen_data_get(key, default)`	In-memory data shared between screens.
`clipboard_copy(text)`	Copy text to the clipboard; needs a user tap, no Android permission.
`vibrate_pannu(ms)`	Vibrate 1-5000 ms; declares the `VIBRATE` permission.

A.2 Configuration functions (top level only)

Function	Purpose
`app_uruvakku("Name")`	App name (1-60 characters).
`app_id_set("com.example.app")`	Application id: lowercase letters, digits and underscores in dot-separated parts; no Java reserved words.
`app_version_set("1.0.0")`	Version as MAJOR.MINOR.PATCH.
`app_theme_set("dark")`	"light" or "dark".
`app_start_screen("home")`	Which screen opens first (defaults to the first declared).

A.3 Common error messages

Message	Cause and fix
Unknown function 'text_kaatu'. Did you mean 'text_kaattu'?	Typo; the suggestion comes from the list of known names.
Variable 'x' is not declared.	Declare it first with `mari x = ...`.
'text_kaattu' draws UI and can only be used inside a screen block.	Inside functions use `message_kaattu` or `toast_kaattu`.
Cannot use '+' between text and number.	Convert with `str(...)`.
Handler 'x' must take no parameters.	Event handlers are called with no arguments.
Screens cannot change global 'n' while drawing	Move the assignment into an event-handler function.

Indentation must be a multiple of 4 spaces	Use four spaces per level, never tabs.
Program ran too long (more than 1000000 steps)	Probably an endless loop.

A.4 Running the project

Commands

```
make setup        # install backend and frontend dependencies
make serve        # build the Studio and serve everything at http://127.0.0.1:8000
make test         # backend (pytest) + runtime (node --test) + frontend (tsc + vitest)
make e2e          # real backend + real Chromium end-to-end test

# Compile a program from the command line
python -m apppanrom.cli compile examples/vanakkam.apn
# Generate an Android project, then build it
python -m apppanrom.cli android-project examples/vanakkam.apn /tmp/myapp
cd /tmp/myapp && gradle --no-daemon assembleDebug
# Create a release keystore (prompts for a password)
python -m apppanrom.cli make-keystore ~/keys/myapp.jks --alias release
```

B Appendix: Glossary

Term	Meaning
AST	Abstract syntax tree: the parsed structure of a program as nested nodes.
IR	Intermediate representation: the validated JSON description of an app that all renderers consume.
Lexer / tokenizer	The first compiler stage; turns text into tokens.
Recursive descent	A parsing technique with one function per grammar rule.
Runtime	The JavaScript interpreter and renderer shared by the preview and the APK.
View tree	The plain-object description of the current screen built by the runtime before it becomes HTML.
Handler	A zero-argument function connected to an event such as a button click.
Diagnostic	A compiler message with severity, position and suggestion.
APK / AAB	Android Package / Android App Bundle: installable app file / Play Store publishing format.
Gradle	The build tool used by Android projects.
WebView	An Android component that displays web content inside an app.
CSP	Content-Security-Policy: a rule set restricting what a page may load or run.
Sandbox (iframe)	A browser mechanism that isolates embedded content from its parent.
Tanglish	Tamil written in English letters.
mari	"Variable"; declares a variable.
enna_na / illana_enna / illana	"If" / "else if" / "else".
ovvoru	"Each"; the for-each loop.
irukkura_varai	"As long as it exists"; the while loop.
summa	"Just" / "nothing"; the no-operation statement.

kaattu	"Show" (as in `text_kaattu`, `toast_kaattu`).
uruvakku	"Create" (as in `screen_uruvakku`).
pannu	"Do" (as in `vibrate_pannu`).